

SORRISO CONECTADO

KENJI FERNANDES WAKABAYASHI - RM 568156

MURILO MARQUES CABRAL - RM 568224

PAULO HENRIQUE DA SILVA KIAN - RM 563343

Superior de Tecnologia em Análise e Desenvolvimento de Sistemas

FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA (FIAP)

2025

Sumário

Introdução	2
Objetivo.....	2
Sprint 1	3
Software engineering and business model.....	3
Front-end design engineering	3
Domain driven design using java	3
Computational thinking using python.....	4
Building relational database	4
Artificial intelligence e chatbot.....	6
Sprint 2	10
Software engineering and business model.....	10
1. Levantamento e Especificação de Requisitos (RF e RNF)	10
2. Elaboração do Business Model Canvas (BMC).....	13
3. Elaboração do Mapa de Empatia	14
4. Elaboração do Diagrama de Caso de Uso	14
Domain driven design using java	18
Building relational database	19
Sprint 3	20
Software engineering and business model.....	20
Domain driven design using java	21
Building relational database	28
Sprint 4	30
Software engineering and business model.....	30
Domain driven design using java	34

INTRODUÇÃO

Este é um documento oficial produzido por alunos do curso de Superior de Tecnologia em Análise e Desenvolvimento de Sistemas da Faculdade de Informática e Administração Paulista (FIAP).

Será registrado neste documento o projeto “Sorriso Conectado”, em prol do desafio proposto pela FIAP junto da instituição Turma do Bem que será desenvolver uma ferramenta integrada de gestão de atendimentos capaz de organizar, direcionar e controlar de forma eficiente todos os contatos recebidos pelos diferentes públicos com os quais a Turma do Bem se relaciona, sendo eles dentistas, voluntários, beneficiados dos programas sociais da Turma do Bem, doadores, pessoas que solicitam serviços da Turma do Bem.

No momento até então, a Turma do Bem recebe contatos por diversos canais, sem ter um focado, sendo os principais meios de contatos o WhatsApp e E-mails, e a dificuldade enfrentada é justamente o controle desses contatos que compromete a agilidade nas respostas. O projeto “Sorriso Conectado” será desenvolvido e documentado pelos alunos Kenji Fernandes Wakabayashi, Murilo Marques Cabral e Paulo Henrique da Silva Kian, que utilizarão do seu conhecimento e aprendizado durante as aulas do curso, juntamente com o auxílio de seus professores.

OBJETIVO

O projeto “Sorriso Conectado” será um sistema CRM (Gestão de Relacionamento com o Cliente), aplicado para melhorar todas as experiências, relacionamentos e meios de contato da Turma do Bem com seus clientes, da forma mais eficiente e simples possível. Um sistema CRM é geralmente utilizado nos setores de vendas, marketing e atendimento, centralizando dados de contato, histórico de compras, e-mails e ligações, entregando como resultado uma visão completa dessas informações em um serviço totalmente personalizado.

A Turma do Bem atualmente sofre com o problema da perda de informações relevantes de seus clientes e a dificuldade de contato com eles, e um sistema CRM aplicado e personalizado para suas necessidades pode aliviar essa dor da Turma do Bem, concentrando todos os contatos em um meio só, categorizado de acordo com a finalidade de cada conversa, assim não haverá confusão na hora de coletar e interpretar dados importantes.

Primeiramente o sistema do Sorriso Conectado irá entender qual a intenção de cada pessoa em entrar contato com a Turma do Bem, se é um dentista a fim de participar e ajudar nos serviços da Turma do Bem, se é um doador, ou se é uma pessoa necessitada dos serviços da Turma do Bem, entre outros casos. Após isso, o sistema irá direcionar cada pessoa para uma conversa específica para aquele assunto e então começará a coletar os dados relevantes para aquela finalidade, dados como nome, CPF, gênero, localização, CNPJ se é uma empresa entrando em contato, entre outros dados importantes. Esses dados coletados serão transformados em informações e armazenados no sistema de modo organizado, para assim facilitar a visualização de cada caso e auxiliar na tomada de decisões da Turma do Bem.

SPRINT 1

Em cada sprint terá todas as entregas relativas das disciplinas e suas propostas para esse desafio, sendo a Sprint 1 as entregas das primeiras propostas, e assim seguindo com prazos diferentes, até a Sprint 4.

SOFTWARE ENGINEERING AND BUSINESS MODEL

Foi organizado um backlog do projeto, contendo os processos de cada sprint e as histórias de usuário de cada cliente da Turma do Bem, que condiz com o objetivo do projeto Sorriso Conectado que foi anteriormente escrito neste documento.

Além disso, foi montando um pitch e gravado um vídeo de apresentação prévia da solução, roteirizado de acordo com a proposta do Sorriso Conectado.

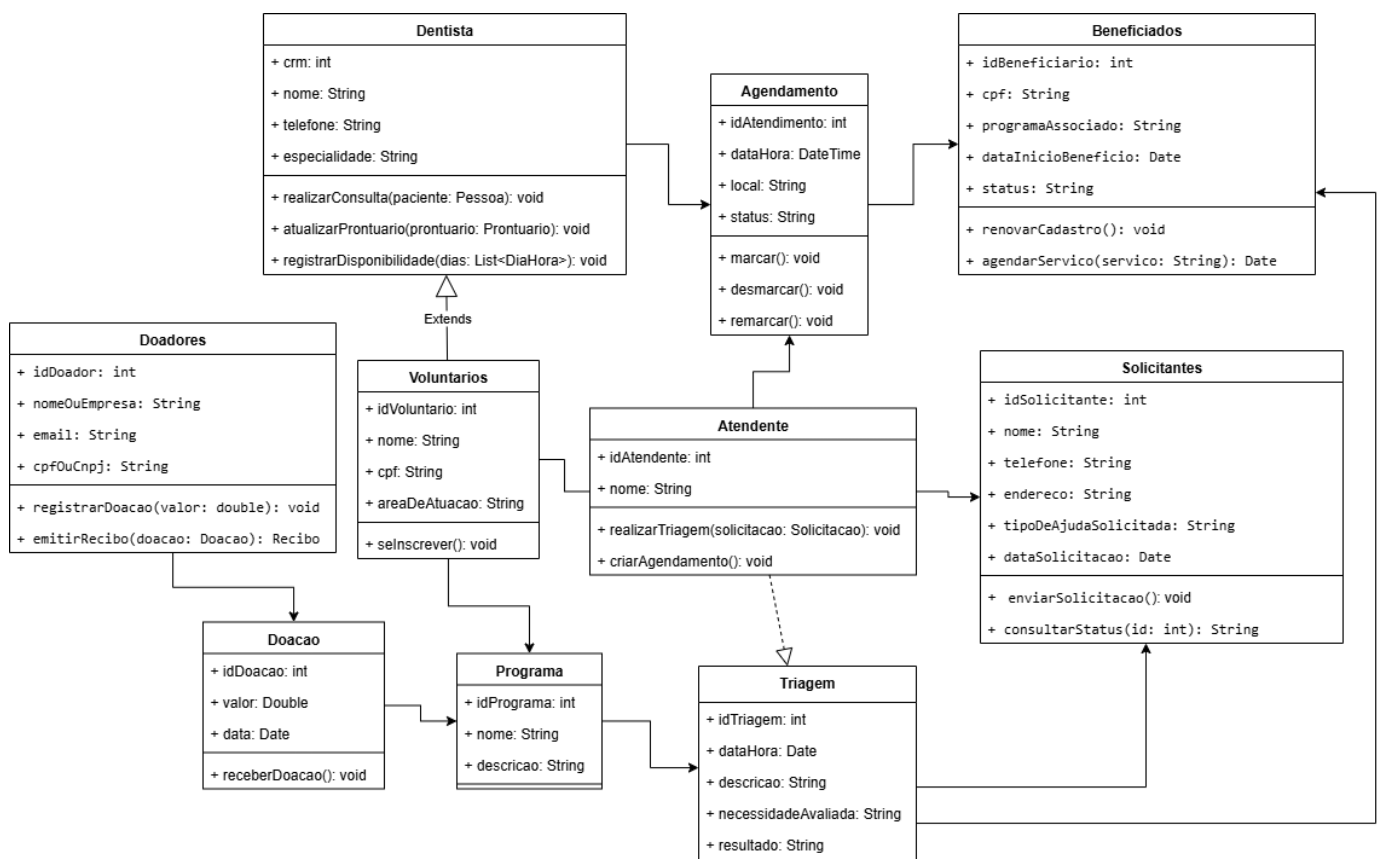
Acesso ao [Trello](#) e ao [vídeo](#) e [apresentação](#) do pitch.

*Obs: “Ctrl + Clique esquerdo” nas palavras em azul (Hyperlink).

FRONT-END DESIGN ENGINEERING

DOMAIN DRIVEN DESIGN USING JAVA

Este Diagrama de Classes atua como o blueprint do "Sorriso Conectado", um sistema CRM fundamental para centralizar e otimizar o relacionamento da Turma do Bem com seus diversos públicos. A estrutura reflete diretamente o desafio de gerenciar, direcionar e controlar eficientemente todos os contatos. O modelo foi organizado para que o sistema possa categorizar cada público de acordo com sua intenção, seja um dentista, um doador, ou uma pessoa necessitada dos serviços, e coletar dados específicos para cada finalidade.



O fluxo de trabalho é gerido por três classes principais: Atendente, Triagem e Agendamento. A Triagem é a peça-chave para o direcionamento, pois o Atendente utiliza o método realizarTriagem() para avaliar a necessidadeAvaliada e o resultado, categorizando o contato conforme a finalidade da conversa. A classe Agendamento formaliza o serviço, conectando o Dentista/Voluntário ao Beneficiado e controlando o ciclo do atendimento com métodos como marcar() e desmarcar().

As classes Programa e Doacao se integram para gerir os projetos sociais e as contribuições financeiras, respectivamente. Em essência, este diagrama traduz o projeto "Sorriso Conectado" em uma estrutura de dados robusta, pronta para transformar contatos dispersos em informações organizadas que apoiam a tomada de decisões da Turma do Bem.

As classes Dentista e Voluntários representam o corpo clínico. O Dentista é a entidade base, com atributos essenciais como crm e especialidade, e métodos para gerenciar a consulta, como realizarConsulta() e atualizarProntuario(). A classe Voluntários estende Dentista, herdando essas funcionalidades e adicionando o controle de areaDeAtuacao e a possibilidade de seInscrever(), refletindo o apoio voluntário da Turma do Bem.

A classe Programa é o catálogo de serviços oferecidos pela Turma do Bem, com nome e descricao. Ela é fundamental, pois está ligada tanto à Triagem (para onde o solicitante será direcionado) quanto aos Beneficiados (quem está efetivamente em um programa).

A Doacao e Doadores lidam com o lado financeiro e de apoio. A classe Doadores registra a origem da contribuição, seja uma pessoa ou uma empresa (nomeOuEmpresa, cpfOuCnpj), e tem a responsabilidade de registrarDoacao() e emitirRecibo(). A classe Doacao em si registra o histórico de cada transação, capturando o valor e a data.

COMPUTATIONAL THINKING USING PYTHON

BUILDING RELATIONAL DATABASE

A ONG “Turma do Bem” tem como missão oferecer cuidado odontológico gratuito a pessoas em situação de vulnerabilidade social. Atualmente, a gestão das informações sobre atendimentos, voluntários, pacientes e doações é feita de forma manual, dificultando o controle, a análise de dados e o planejamento de ações estratégicas.

O objetivo da solução proposta é criar um sistema informatizado com banco de dados relacional, que permita:

- Registrar e organizar informações de pacientes, voluntários, atendimentos e doações;
- Controlar os agendamentos de consultas odontológicas;
- Monitorar os recursos e insumos disponíveis para os atendimentos;
- Gerar relatórios e estatísticas para tomada de decisão da ONG.

Com isso, a ONG terá mais eficiência operacional, garantindo que mais pessoas possam ser atendidas com qualidade.

Para atender ao objetivo acima, o banco de dados deverá armazenar informações sobre:

Pacientes

- Dados pessoais: nome, CPF, data de nascimento, endereço, telefone, e-mail;
- Histórico de atendimentos odontológicos;
- Situação social (opcional, para entender o perfil dos atendidos).

Voluntários

- Dados pessoais: nome, CPF, formação/área de atuação, telefone, e-mail;
- Disponibilidade de horários;
- Registro de atendimentos realizados.

Consultas/Atendimentos

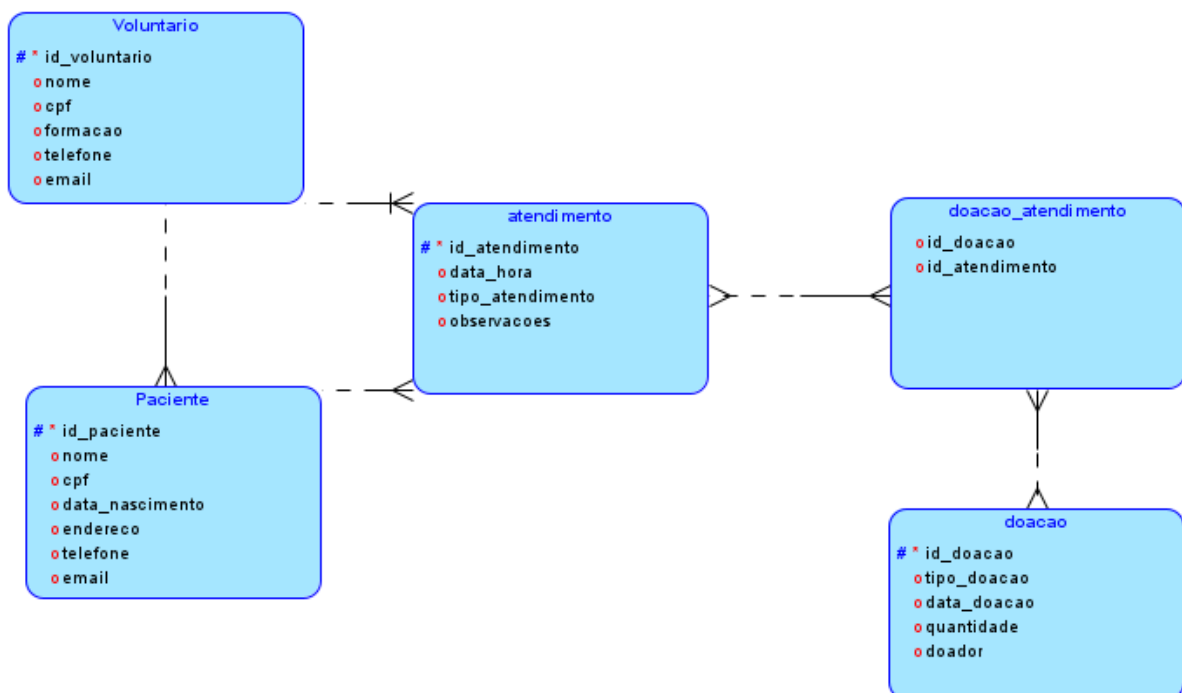
- Data e hora do atendimento;
- Profissional responsável;
- Paciente atendido;
- Tipo de atendimento (ex.: limpeza, restauração, extração);
- Observações sobre o atendimento.

Doações e Recursos

- Tipo de doação (financeira, insumos odontológicos, equipamentos);
- Data da doação;
- Quantidade/valor;
- Doador responsável.

Relacionamentos Entre Entidades

- Cada paciente pode ter múltiplos atendimentos;
- Cada voluntário pode atender múltiplos pacientes;
- Cada atendimento deve estar vinculado a um paciente e a pelo menos um voluntário;
- Doações podem ser realizadas por doadores individuais ou empresas e podem ser utilizadas em múltiplos atendimentos.



A solução de banco de dados permitirá que a “Turma do Bem” organize e controle todas as informações essenciais para a sua operação. A estrutura relacional facilitará consultas rápidas, geração de relatórios e tomadas de decisão, contribuindo diretamente para a melhoria do atendimento à população e para o gerenciamento eficiente dos recursos da ONG.

ARTIFICIAL INTELLIGENCE E CHATBOT

Como primeira Sprint, foi elaborado um exemplo de como o ChatBot do WatsonX pode ser útil para realizar alguns diálogos simples e agilizar alguns processos com os clientes da Turma do Bem.

Foi utilizado como exemplo um cenário onde uma pessoa que necessita a ajuda da Turma do Bem recorre ao atendimento da instituição, o atendente humano da Turma do Bem terá que fazer algumas perguntas básicas para conseguir encaminhar o usuário para a melhor solução possível do seu problema, porém essas perguntas padrões podem ocupar um bom tempo do atendente humano, tempo esse que se somado com outros atendimentos a usuários que respondem também as mesmas perguntas básicas do atendente, poderia ser transformado em mais atendimentos no dia ou em outras tarefas produtivas.

O nosso ChatBot, SorrisoBot, fará o trabalho de realizar essas perguntas básicas iniciais ao usuário, agilizando esse processo do atendente humano da Turma do Bem e assim solucionando em parte o problema apresentando anteriormente, recolhendo dados como local, gênero e renda mensal do usuário, e assim enviando tudo ao banco de dados do nosso sistema, deixando uma visualização completa do usuário para o atendente humano, que estará pronto para realizar algumas perguntas mais complexas ao usuário e assim completar seu cadastro para ajudá-lo.

Aplicado com uma linguagem simples não muito formal para uma interpretação fácil ao usuário, com respostas curtas, objetivas e claras, sem muita complexidade ou nível técnico elevado.

Lista de Intenções

- 1- #FAQ_localidade // Saber até onde a Turma do Bem opera
- 2- #Ajuda_iniciar_ajuda // Reconhece que o usuário é uma pessoa que necessita da ajuda da Turma do Bem
- 3- #Ajuda_genero // Reconhece o gênero do usuário
- 4- #Ajuda_renda // Pergunta qual a renda mensal da pessoa em reais
- 5- #Ajuda_beneficiado // Pergunta quem necessita de ajuda, se é o próprio usuário quem responde ou se é pra outra pessoa (Filho e afins)
- 6- #Ajuda_local // Pergunta onde a pessoa mora

Exemplos de intenção (perguntas/respostas ao sistema)

- 1)
 - a) onde vocês operam
 - b) até onde vocês trabalham
 - c) aonde a turma do bem está

- d) em que local posso encontrar a turma do bem
 - e) qual região a turma do bem atende
- 2)
- a) preciso de ajuda
 - b) procuro ajuda
 - c) necessito dos serviços da turma do bem
 - d) quero ajuda
 - e) quero um dentista
- 3)
- a) masculino
 - b) feminino
 - c) homem
 - d) mulher
 - e) mulher trans
- 4)
- a) 1000 reais
 - b) R\$ 400
 - c) 300 conto
 - d) reais 500
 - e) 7000 R\$
- 5)
- a) eu
 - b) meu filho
 - c) para minha filha
 - d) para mim
 - e) minha mãe
- 6)
- a) São Paulo
 - b) Zona Sul SP
 - c) Rio de Janeiro
 - d) ZL
 - e) Santana

Exemplos de perguntas/respostas do sistema

- 1)
- a) Operamos no Brasil todo e em mais 12 países
 - b) Em todo o Brasil
 - c) Em toda a América do Sul e países da América espanhola
 - d) Em toda São Paulo e os demais estados do Brasil
 - e) Trabalhamos em todo território nacional
- 2)
- a) Entendi, você busca os serviços da Turma do Bem!
 - b) A Turma do Bem irá te ajudar!
 - c) Compreendi, vamos te ajudar.

- d) Certo! Você busca ajuda da Turma do Bem.
 - e) Vou te ajudar!
- 3)
- a) Qual o seu gênero?
 - b) Qual o seu sexo?
 - c) Informe seu gênero
 - d) Poderia me informar seu gênero?
 - e) Pode me dizer qual o seu sexo?
- 4)
- a) Me diga qual a sua renda mensal em reais
 - b) Qual a sua renda mensal em reais?
 - c) Poderia me dizer aproximadamente a sua renda mensal em reais?
 - d) Informe sua renda mensal.
 - e) Pode me informar a sua renda mensal em reais?
- 5)
- a) Para quem seria a ajuda?
 - b) Os serviços da Turma do Bem serão para você, ou para outra pessoa?
 - c) Pode me dizer quem precisa da ajuda da Turma do Bem (Você ou outra pessoa)
 - d) Para quem você busca os serviços da Turma do Bem? (Você ou outra pessoa)
 - e) A Turma do Bem irá ajudar você ou outra pessoa? (Filho, Mãe, Amigos, etc)
- 6)
- a) Poderia me dizer onde você mora?
 - b) Pode me informar seu endereço?
 - c) Pode me dizer onde você mora?
 - d) Aproximadamente, aonde você reside?
 - e) Por favor, informe o seu endereço.

Lista de Entidades

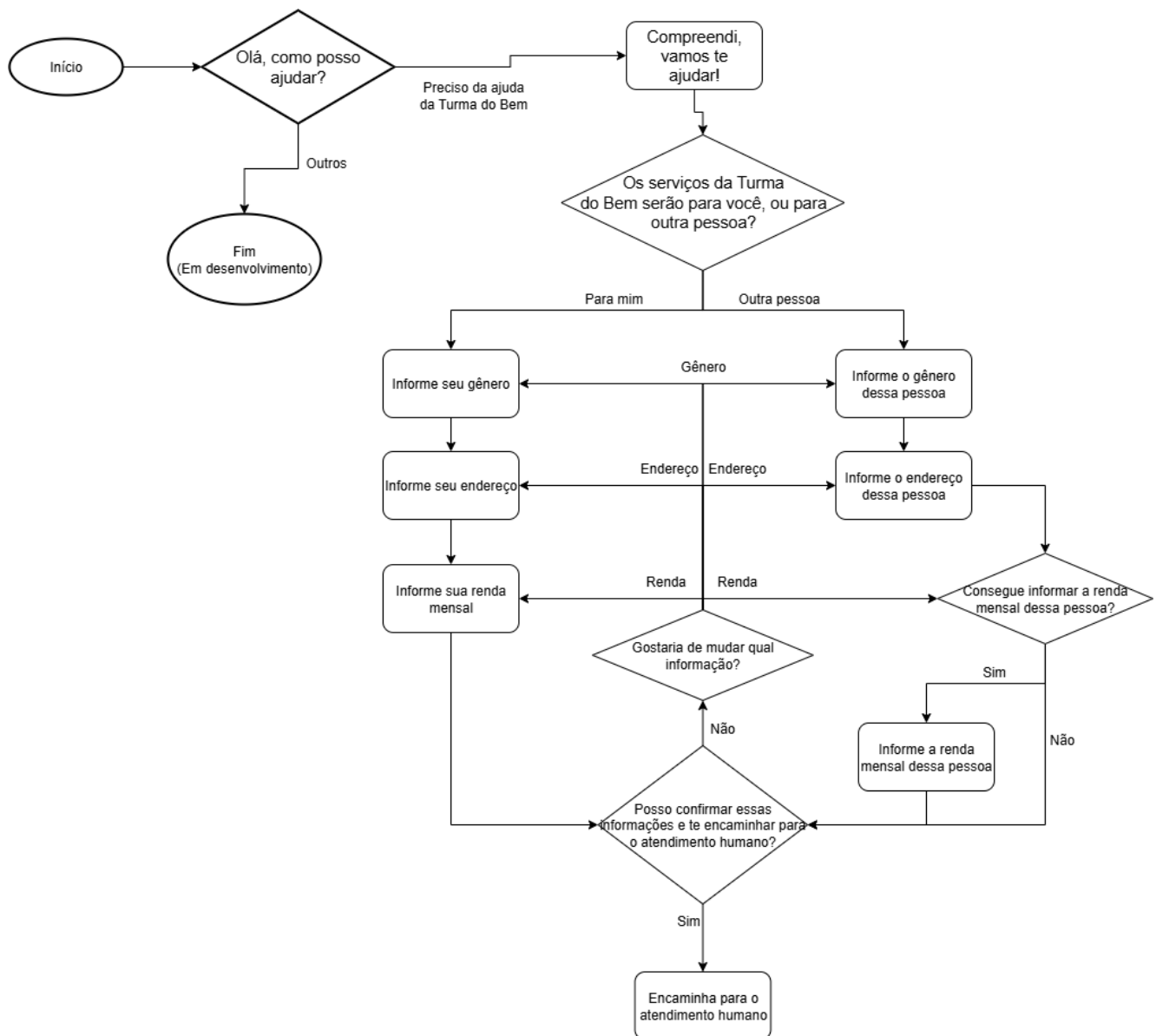
- @cidade // São Paulo, Rio de Janeiro, Salvador, SP, RJ, SA, etc // Para detectar em que cidade o usuário se encontra, pode ser usado para descobrir onde o usuário mora ou responder onde a Turma do Bem opera (Ex: A Turma do Bem opera em São Paulo?; Moro em Rio de Janeiro).
- @endereço // Zona Leste, Zona Sul, Zona Oeste, Zona Norte, ZL, ZS, ZO, ZN, Vila Mariana, Santana, Alto do Ipiranga, Sacomã, Vila Guilhermina, Consolação, Botafogo, Copacabana, Heliópolis, Morumbi, Pinheiros, Av. Paulista, Centro, \d{5}\-\d{3} // Para saber o local mais aproximado em que o usuário se encontra.
- @sys-currency // 1000, 500, 343, 21, 03, 1, 3, 5,6 9,9 19,99// Para ler valores, como a renda mensal do usuário.
- @genero // masculino, feminino, homem, mulher, menino, menina, homem-trans, mulher-trans, homem-cis, mulher-cis, não binário, etc // Para identificar o gênero do usuário.

- @destinatario // Eu, mim, meu filho, filha, pai, minha mãe, minha avó, vó, meu sobrinho, meu amigo, para minha amiga, para um conhecido // Para ajudar a descobrir quem precisará dos serviços da Turma do Bem.
- @confirmacao // Sim, Não, Ss, NN, nao, sim, positivo, negativo, pode ser, acho que não // Para capturar a confirmação do usuário se necessário.

Exemplo de diálogo:

1. **SorrisoBot:** Olá, como posso ajudar?
2. **Usuário:** Olá, preciso da ajuda da Turma do Bem
3. **SorrisoBot:** Compreendi, vamos te ajudar!
4. **SorrisoBot:** Os serviços da Turma do Bem serão para você, ou para outra pessoa?
5. **Usuário:** Seria para mim mesmo
6. **SorrisoBot:** Poderia me informar seu gênero?
7. **Usuário:** Feminino
8. **SorrisoBot:** Pode me informar seu endereço?
9. **Usuário:** R. Vicente Rodrigues da Silva, 506 – Piratininga – Osasco
10. **SorrisoBot:** Pode me informar sua renda mensal em reais?
11. **Usuário:** Mais ou menos R\$ 1.500
12. **SorrisoBot:** Ok! Posso confirmar essas informações e te encaminhar para o nosso atendimento humano?
13. **Usuário:** Sim.
14. **SorrisoBot:** Encaminhando para o atendimento humano...

Fluxograma SorrisoBot



SPRINT 2

SOFTWARE ENGINEERING AND BUSINESS MODEL

Acesso ao [Trello](#)

Nesta etapa, o foco foi na análise aprofundada do negócio e na especificação formal do sistema, garantindo que as regras de negócio da Turma do Bem estivessem alinhadas aos requisitos técnicos do 'Sorriso Conectado'. Utilizamos ferramentas de engenharia de software e design thinking para refinar a visão do projeto.

1. LEVANTAMENTO E ESPECIFICAÇÃO DE REQUISITOS (RF E RNF)

Realizamos o levantamento completo dos Requisitos Funcionais (RF) e Não-Funcionais (RNF).

Requisitos Funcionais				
Código	Requisito	Descrição	Prioridade	Regra de Negócio
RF001	Botão Home	Um botão em cada página em local estratégico (Cantos) para o usuário conseguir acessar a área inicial do sistema	Essencial	O usuário deve acessar a área inicial do sistema.
RF002	Efetuar Cadastro	Cadastro do usuário para conseguir acesso ao login e acesso a plataforma Sorriso Conectado	Essencial	O usuário deve registrar seu nome de usuário, senha, e-mail e opcionalmente telefone.
RF003	Efetuar Login	Permitir que o usuário se conecte ao sistema	Essencial	O usuário deve informar seu nome de usuário ou e-mail e a senha anteriormente registrado para identificação.
RF004	Esqueci a Senha	Permitir que o usuário recupere a senha por meio de e-mails ou SMS pelo telefone previamente registrado	Essencial	O usuário deve ter feito o cadastro com um e-mail e telefone próprios e dedicados para essa situação.
RF005	Realizar Doação	Um botão ou campo clicável onde o usuário possa ser direcionado para o campo de doações da Tdb	Importante	A Tdb deve haver um site ou plataforma para receber doações de forma virtual.
RF006	Entrar em contato	Deve haver um formulário, propriamente programado e embutido na plataforma onde o usuário consiga escrever informações pessoais necessárias, assunto e a mensagem sobre o contato necessário, ou pode ser um botão ou campo clicável para direcionar o usuário a um formulário já feito, tipo Google Forms	Importante	O usuário deve ser capaz de escrever no formulário, onde todos os campos são obrigatórios.
RF007	Site da Tdb	Um campo onde o usuário consiga ser direcionado para o site oficial da Turma do Bem	Importante	O usuário deve conseguir interagir com esse campo para ser direcionado.

Requisitos Não Funcionais				
Código	Requisito	Descrição	Prioridade	Regra de Negócio
RNF001	Conectividade	É necessário conexão com a internet para acessar o sistema.	Essencial	O usuário deve estar conectado com a internet para ter acesso

				ao sistema e conseguir usá-lo propriamente
RNF002	Armazenamento de dados em um Banco de Dados	É necessário o armazenamento de dados em um banco de dados.	Essencial	O sistema só consegue processar, extrair e importar dados com um banco de dados apropriado.
RNF003	Criptografia dos dados	É necessária uma criptografia quando armazenar os dados no BD	Importante	O BD deve haver criptografia para manter uma segurança dos dados armazenados, pois serão dados de pessoas públicas e da Tdb.
RNF004	Usuário já deve ter login no sistema	O usuário deve haver um cadastro no sistema.	Essencial	O usuário não deve conseguir acessar o sistema sem se identificar
RNF005	Validação de CPF/CNPJ	O CPF ou CNPJ informado ao sistema pelo usuário durante seu cadastro deve ser validado.	Importante	Se o CPF ou CNPJ do usuário não for válido, ele não deve conseguir se cadastrar no sistema.
RNF006	Validação de E-mail	O E-mail informado ao sistema pelo usuário durante seu cadastro deve ser validado	Importante	Se o E-mail do usuário não for válido, ele não deve conseguir se cadastrar no sistema.
RNF007	Validação de Senha	A senha informada ao sistema pelo usuário durante seu cadastro deve ser validada	Importante	A senha do usuário deve conter no mínimo 8 caracteres, um número, uma maiúscula, uma minúscula e um caractere especial.
RNF008	Confirmação de E-mail	O sistema deve enviar um código para e-mail que o usuário informar para confirmá-lo.	Importante	Sem o e-mail confirmado, o usuário não conseguirá recuperar a senha caso perda.
RNF009	Confirmação de telefone	O sistema deve enviar um código para o	Importante	O telefone entra como uma opção

		telefone que o usuário informar para confirmá-lo		reserva do e-mail caso o usuário também perca o e-mail.
--	--	--	--	---

2. ELABORAÇÃO DO BUSINESS MODEL CANVAS (BMC)

Desenvolvemos o Business Model Canvas (BMC) para visualizar o projeto sob uma perspectiva de negócio e valor.

O BMC identificou a Proposta de Valor do Sorriso Conectado como sendo a Centralização e Organização dos Dados e Comunicação (resolvendo o problema da perda de informação). Os Segmentos de Cliente foram claramente definidos, abrangendo todos os públicos da TdB (Dentistas, Voluntários, Beneficiados, etc.).

O BMC também mapeou os Recursos Chave (equipe, tecnologias Java/Python) e os Fluxos de Receita (doações e parcerias), alinhando o impacto social à sustentabilidade da plataforma.

Business Model Canvas

Sorriso Conectado



3. ELABORAÇÃO DO MAPA DE EMPATIA

O Mapa de Empatia foi criado para nos aprofundarmos nas necessidades e emoções do nosso principal usuário administrador: o **Atendente da Turma do Bem**.

- O mapa nos permitiu entender o que o Atendente **Vê** (contatos dispersos, planilhas desorganizadas), o que ele **Pensa e Sente** (estresse, frustração com a perda de informação) e o que ele **Ouve** (reclamações de pacientes, pedidos urgentes).
- Isso foi crucial para focar a solução na **facilidade de uso** e na **organização imediata**, garantindo que o CRM resolva a dor operacional do usuário final.

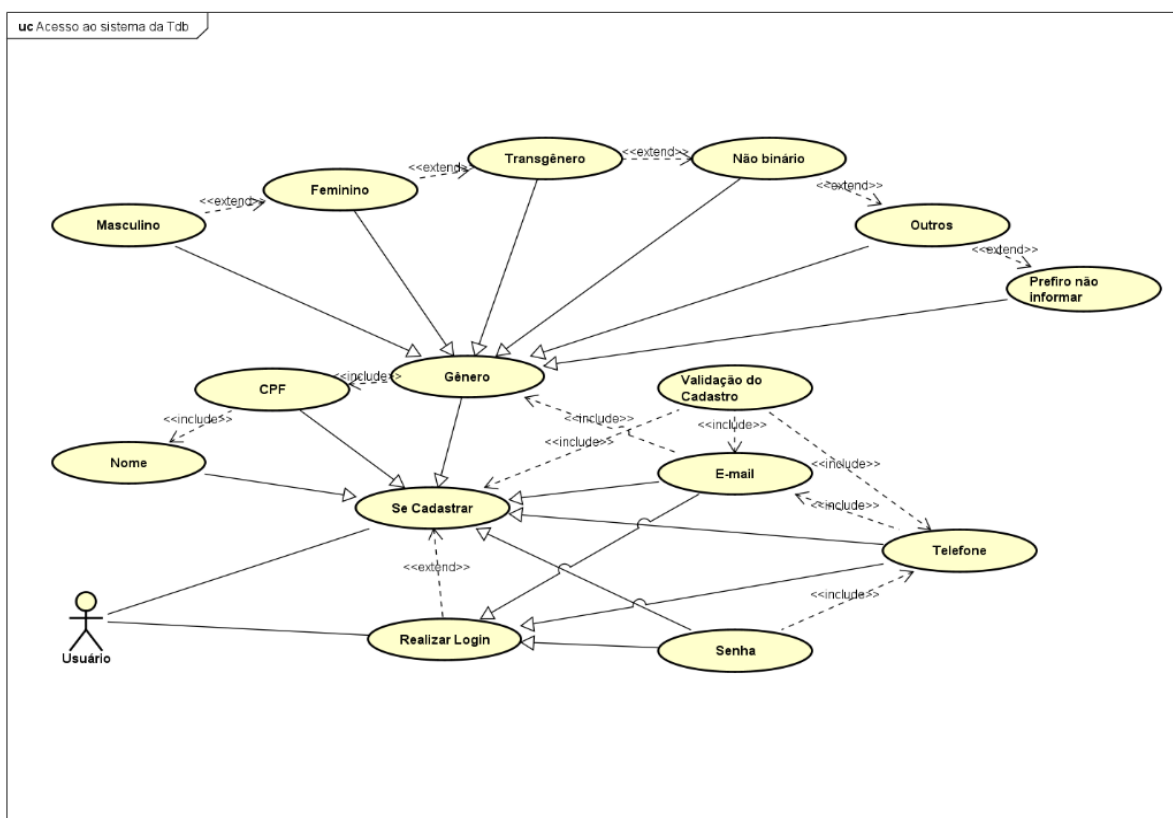


4. ELABORAÇÃO DO DIAGRAMA DE CASO DE USO

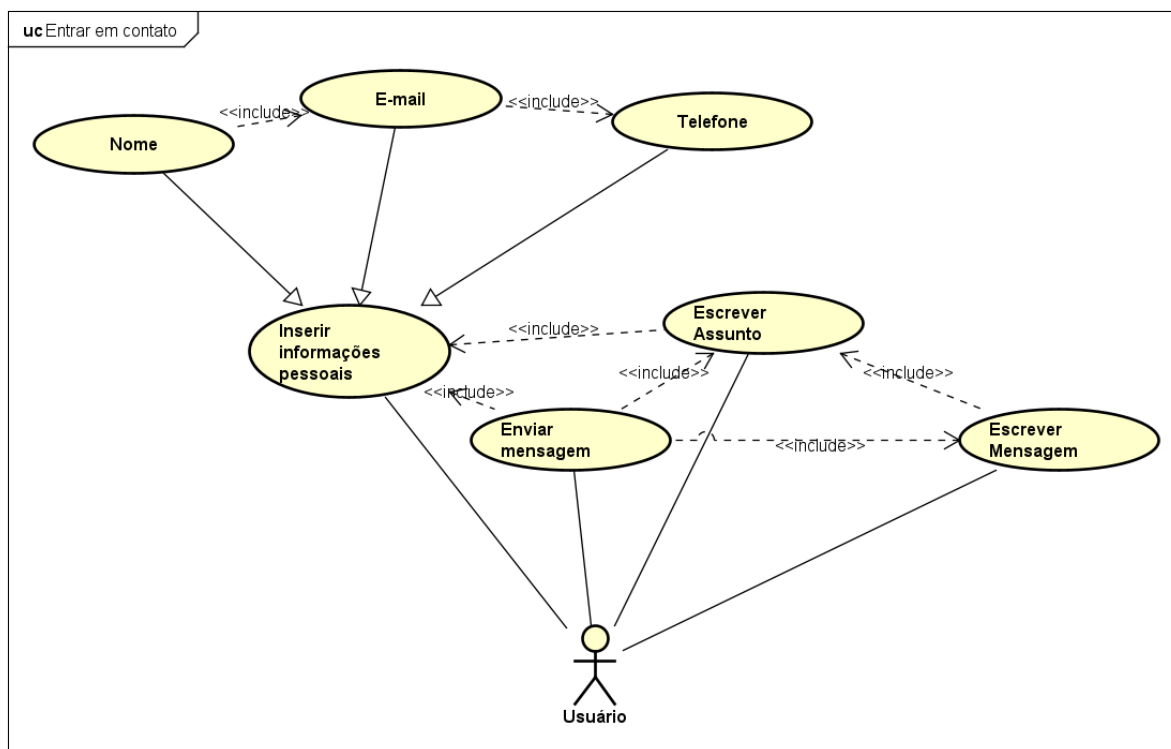
O Diagrama de Caso de Uso descreve as funcionalidades do sistema na perspectiva do usuário, mapeando as interações dos atores (usuários) com os casos de uso (o que o sistema faz).

Três diagramas essenciais foram criados, abrangendo:

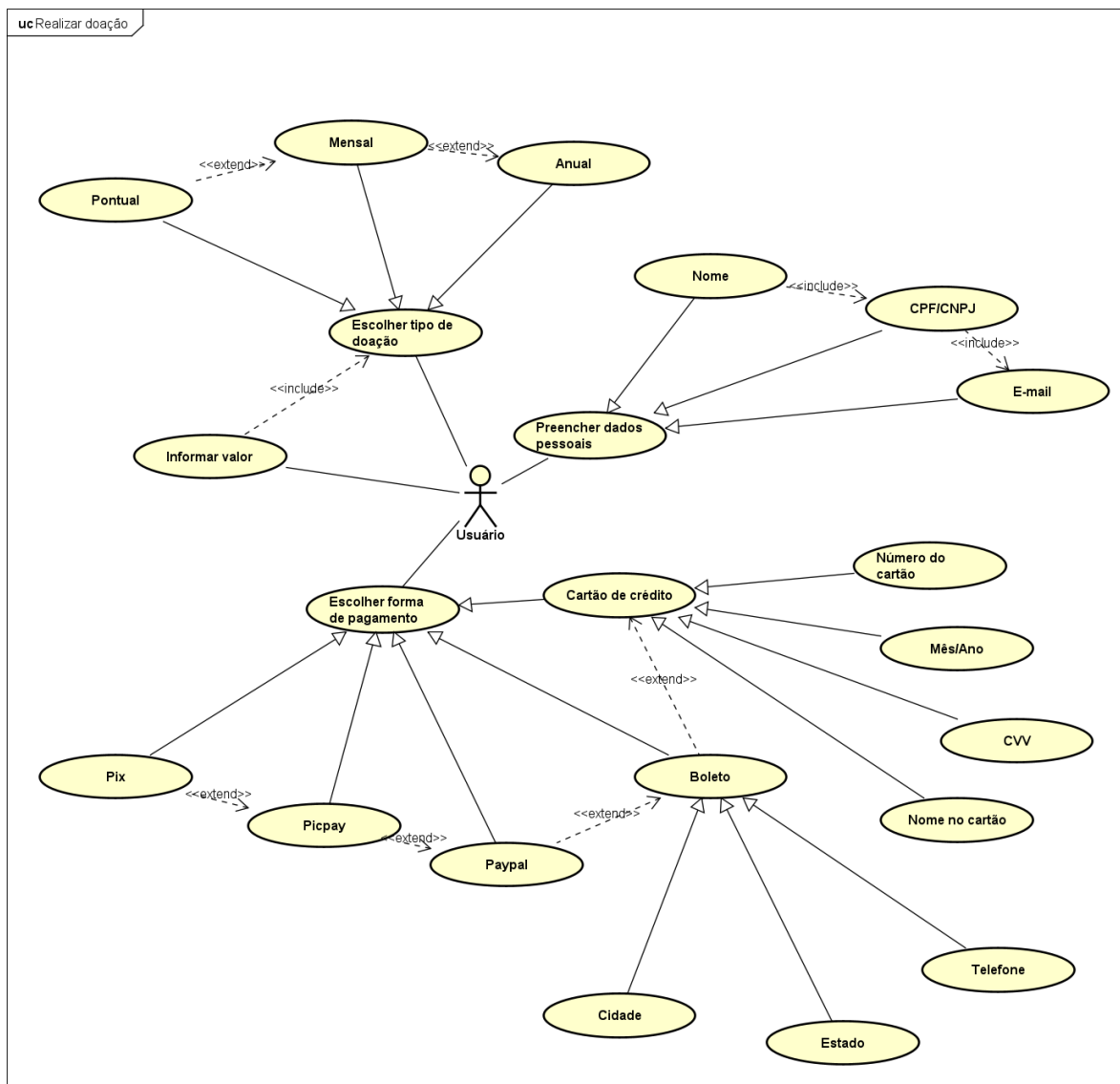
- **Acesso ao Sistema:** Mapeando as interações de **Efetuar Login** e **Efetuar Cadastro**.
- **Entrar em Contato:** Detalhando como o usuário (Solicitante) pode **Entrar em Contato** e **Solicitar Ajuda**.
- **Realizar Doação:** Especificando o fluxo para o **Doador**.



Atributo	Descrição
Nome do Caso de Uso	Acesso ao Sistema (Login e Cadastro)
Atores Envolvidos	Usuário (Visitante, Atendente, Voluntário, Doador)
Fluxo Principal	1. Login: O Usuário insere Nome de Usuário (ou E-mail) e Senha (RF003). O sistema autentica as credenciais, verificando o registro criptografado no banco de dados (RNF003). Se for válido, redireciona o Usuário para a área inicial do sistema (RF001), de acordo com seu perfil.
Fluxo Alternativo	A. Efetuar Cadastro: O Usuário (Visitante) acessa a tela de Cadastro. Insere os dados básicos (RF002). O sistema valida CPF/CNPJ (RNF005) e E-mail (RNF006). Se validado, cria a conta. B. Esqueci a Senha: O Usuário clica em recuperar senha (RF004) e inicia o fluxo de recuperação via E-mail ou Telefone.
Exceções	E1: Credenciais Inválidas: O sistema detecta dados de login incorretos e exibe erro. E2: Falha na Validação (Cadastro): O CPF, CNPJ, E-mail ou Senha não atendem aos critérios de segurança (RNF005, RNF006, RNF007). O sistema notifica o Usuário e impede a criação da conta.
Regras de Negócio Associadas	RF001: O Usuário deve ser redirecionado para a Home. RF003: Permite conexão por Nome de Usuário/E-mail e Senha. RNF007: A senha deve atender aos critérios mínimos de segurança (8 caracteres, maiúsculas, minúsculas, especial).



Atributo	Descrição
Nome do Caso de Uso	Entrar em Contato / Solicitar Ajuda
Atores Envolvidos	Usuário (Visitante ou Solicitante)
Fluxo Principal	1. O Usuário acessa o formulário de Contato ou o Chatbot. 2. O Usuário seleciona o Assunto (ex: Solicitação de Ajuda, Dúvida). 3. O Usuário preenche os dados pessoais (Nome, E-mail) e a mensagem (RF006). 4. O sistema valida os campos obrigatórios e registra a solicitação na fila de atendimento do Atendente. 5. O sistema envia uma notificação ao Atendente para início da triagem.
Fluxo Alternativo	A. Interação via Chatbot: O Usuário dialoga com o Chatbot (IA) que realiza uma pré-triagem (coletando dados como gênero e renda, conforme exemplo) e encaminha o resumo formatado para a fila de atendimento humano.
Exceções	E1: E-mail Inválido: O E-mail informado não passa na validação (RNF006). O sistema impede o envio. E2: Campos Obrigatórios Vazios: O sistema impede o envio e destaca os campos faltantes.
Regras de Negócio Associadas	de RF006: O sistema deve permitir que o Usuário escreva a mensagem e envie as informações necessárias (pessoais, assunto e mensagem). RNF006: O E-mail do Usuário deve ser validado.



Atributo	Descrição
Nome do Caso de Uso	Realizar Doação
Atores Envolvidos	Usuário (Doador)
Fluxo Principal	1. O Doador clica no botão ou campo Realizar Doação (RF005). 2. O sistema direciona o Doador para a plataforma de Doações da TdB (página externa ou formulário interno). 3. O Doador escolhe o valor e o método de pagamento. 4. O Doador finaliza a transação. 5. O sistema registra a Doação e envia o comprovante de pagamento ao Doador.
Fluxo Alternativo	A. Doação Recorrente: O Doador opta pela opção de assinatura mensal, e o sistema registra a recorrência da cobrança. B. Doação Específica: O Doador pode ser direcionado para doar a um Programa específico da TdB (ex: Apolônias do Bem).

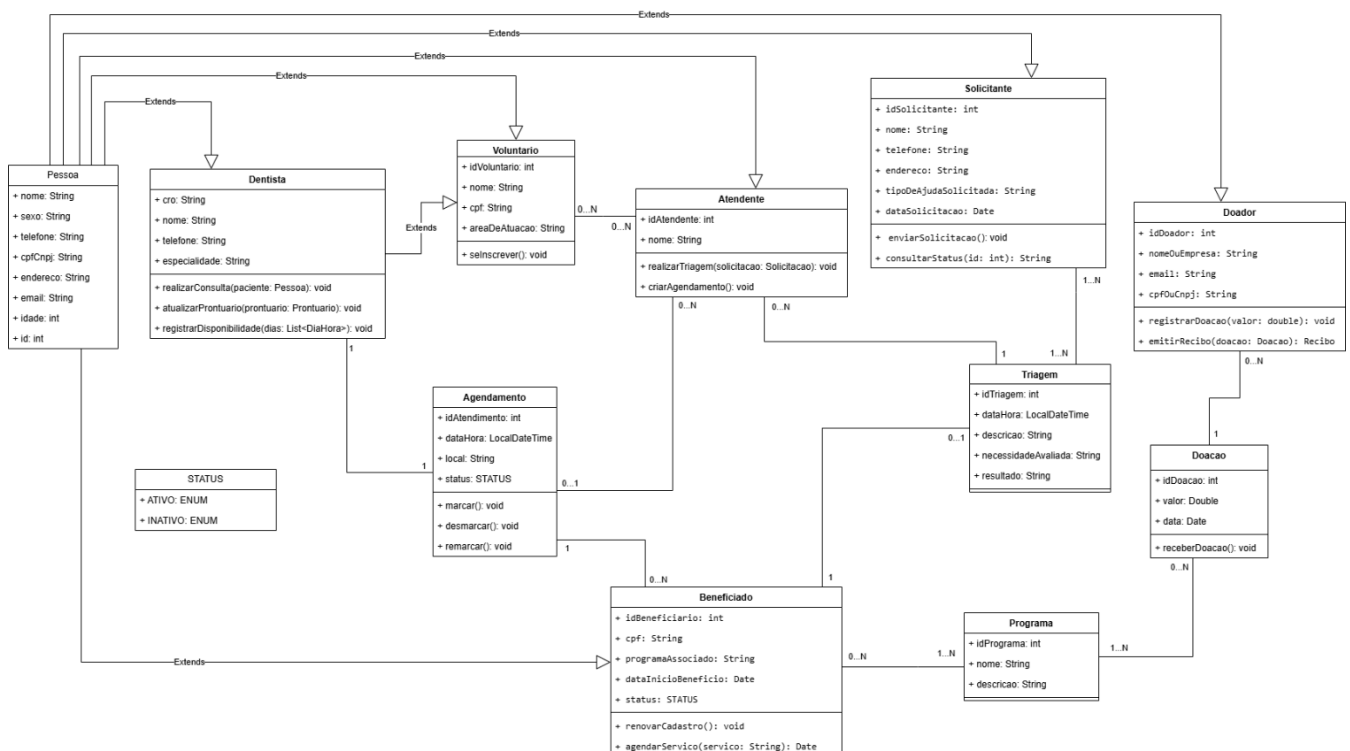
Exceções	E1: Falha na Transação: A transação é recusada pelo gateway de pagamento. O sistema notifica o Doador para que tente novamente.
Regras de Negócio Associadas	RF005: Deve haver um botão ou campo clicável que direcione o Usuário ao fluxo de doações.

DOMAIN DRIVEN DESIGN USING JAVA

O foco desta Sprint foi a criação e configuração do projeto Java em si, estabelecendo a fundação do código que dará vida ao 'Sorriso Conectado'. O projeto foi estruturado em camadas modulares, separando o código em pacotes como entidades e services. A classe Main.java já exibe um menu de inicialização, que delega as funcionalidades de Cadastrar e Agendar aos serviços correspondentes, garantindo que a lógica não fique misturada na tela principal.

Apesar de as classes de entidade ainda estarem em fase inicial, apenas com variáveis e Getters/Setters, os Serviços já traçam o fluxo de negócio. O cadastroService.java é crucial, pois implementa o primeiro passo do Caso de Uso "Entrar em Contato": o método cadastrarSolicitante() coleta dados essenciais como CPF e o tipoDeAjudaSolicitada, dando vida à entidade Solicitante. De forma semelhante, o agendarService.java estabelece o fluxo de marcação de serviços, pedindo informações do cliente e do doutor, preparando o terreno para a futura interação entre Beneficiado, Dentista e Agendamento.

Em paralelo, o Diagrama de Classes foi corrigido e usado como o guia para o código. As melhorias corrigiram o modelo anterior: a relação entre Dentista e Voluntarios foi alterada para Herança, e foram adicionadas as Cardinalidades em todas as conexões, o que torna as regras de negócio explícitas. O modelo também ganhou elementos essenciais para o Java, como a classe Enum Status, que será usada para padronizar os estados de Triagem e Agendamento.

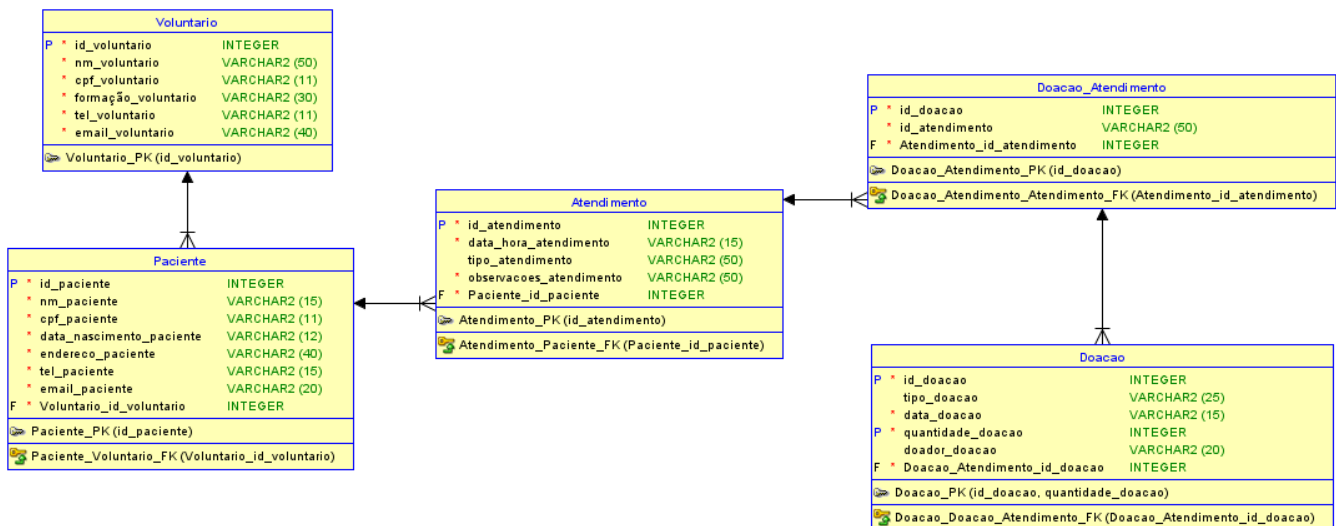


BUILDING RELATIONAL DATABASE

O objetivo central desta Sprint foi a evolução do Modelo Lógico de Dados para o Modelo Relacional. Este modelo serve como o blueprint final e físico, pronto para a criação do banco de dados em qualquer SGBD (Sistema Gerenciador de Banco de Dados). A transição para o Relacional garantiu o rigor na aplicação da Integridade Referencial e a definição exata dos tipos de dados, o que é fundamental para a fase de construção (DDL).

Neste modelo relacional, as regras de negócio complexas do 'Sorriso Conectado' foram traduzidas para o esquema SQL de maneira precisa. A Implementação da Herança (Dentista é um Voluntário) foi resolvida através da replicação da chave primária (id_voluntario) na tabela DENTISTA, estabelecendo esta coluna como Chave Estrangeira (FK) e também como Chave Primária (PK) para a tabela DENTISTA. Esta abordagem garante que o registro do Dentista herde os dados básicos do Voluntário sem duplicidade, mantendo a integridade.

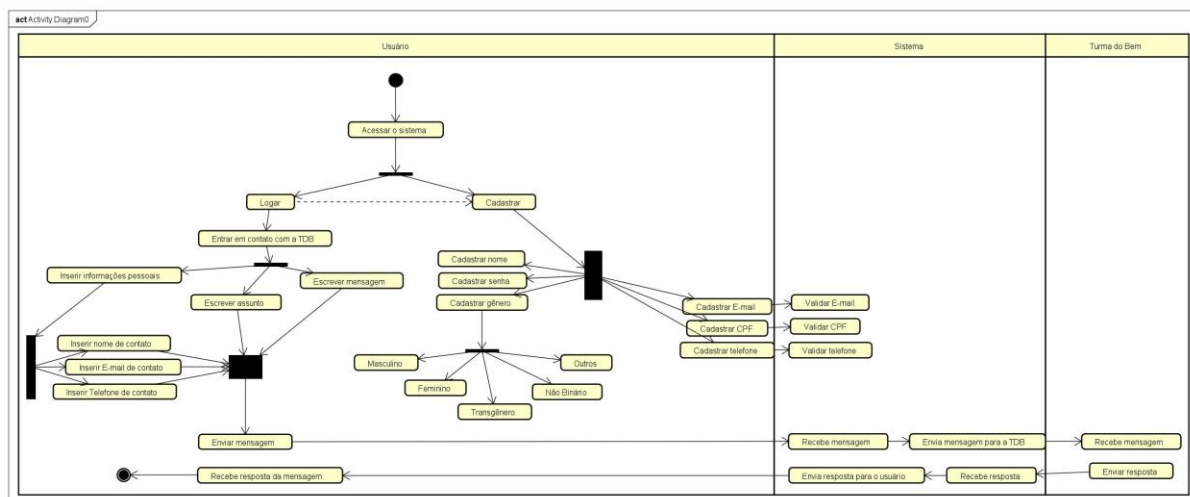
As Associações um-para-muitos (1:N) foram formalmente mapeadas com a inserção de chaves estrangeiras. Campos como id_atendente e id_solicitante foram inseridos nas tabelas TRIAGEM e AGENDAMENTO, o que assegura a rastreabilidade e a referência de quem iniciou ou é responsável por cada registro. Além da definição das Chaves (PK/FK), o Modelo Relacional especificou os tipos de dados para cada coluna, migrando de tipos genéricos para específicos (ex: VARCHAR(255), DATE, DECIMAL), e adicionou restrições (constraints), como a cláusula NOT NULL em campos essenciais.



SPRINT 3

SOFTWARE ENGINEERING AND BUSINESS MODEL

Diagrama de Atividades

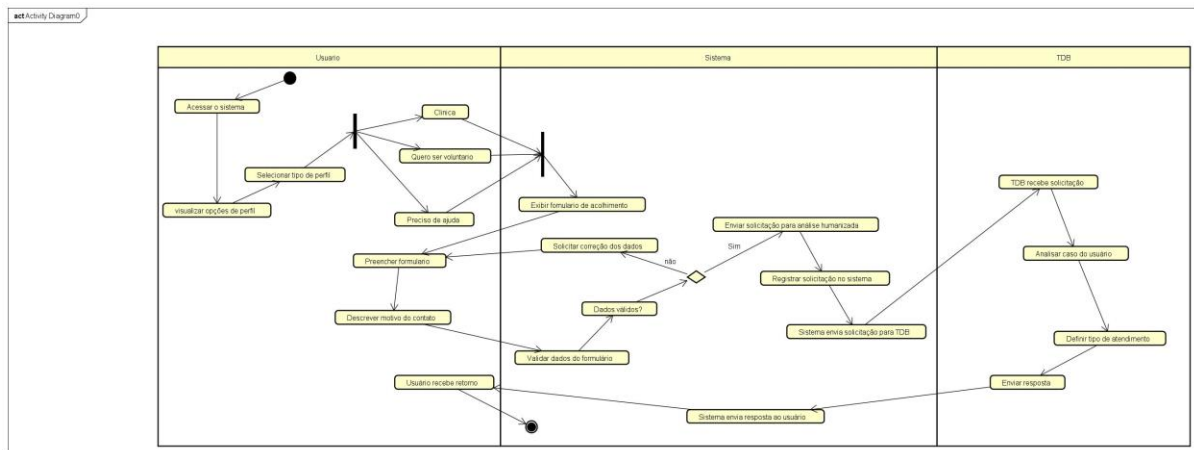


O processo começa quando o usuário acessa o sistema, podendo escolher entre fazer login ou criar uma conta. Caso opte pelo cadastro, ele deve informar dados como nome, senha, gênero, e também informações de contato, como e-mail, CPF e telefone.

Após o preenchimento, o sistema realiza a validação desses dados para garantir que estão corretos, verificando, por exemplo, se o e-mail, o CPF e o telefone são válidos. Com o cadastro concluído ou após o login, o usuário passa a ter acesso às funcionalidades do sistema.

Em seguida, ele pode entrar em contato com a Turma do Bem, preenchendo um formulário com suas informações pessoais, o assunto e a mensagem que deseja enviar. O sistema recebe esses dados e encaminha a mensagem para a instituição.

A Turma do Bem então analisa a solicitação e envia uma resposta. Essa resposta retorna ao sistema, que a apresenta ao usuário. O processo é finalizado quando o usuário recebe esse retorno.



O processo começa quando o usuário acessa o sistema e visualiza as opções de perfil disponíveis. Em seguida, ele escolhe o tipo de perfil que melhor representa sua necessidade, podendo ser clínica, voluntário ou alguém que precisa de ajuda.

Após essa escolha, o sistema exibe o formulário de acolhimento. O usuário então preenche seus dados e descreve o motivo do contato. Em seguida, o sistema valida as informações inseridas. Caso os dados estejam incorretos, o sistema solicita a correção e o usuário deve preencher novamente. Se os dados estiverem válidos, a solicitação é enviada para análise humanizada.

O sistema registra essa solicitação e a encaminha para a Turma do Bem. A instituição recebe o pedido, analisa o caso do usuário e define o tipo de atendimento necessário. Após essa análise, uma resposta é enviada.

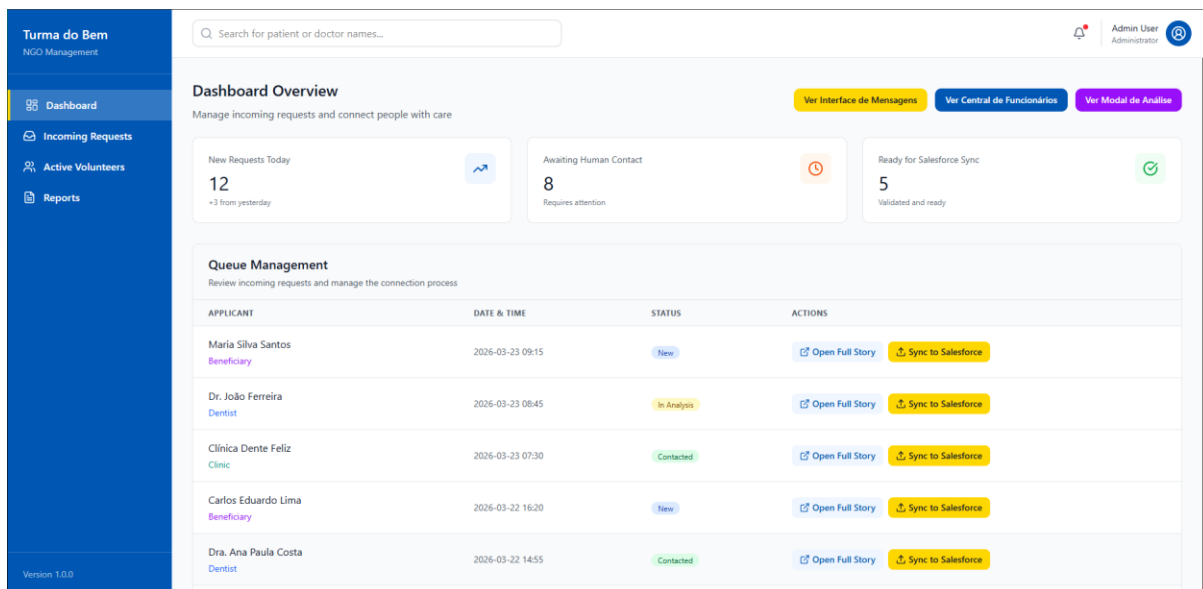
Por fim, o sistema recebe essa resposta e a encaminha ao usuário, que recebe o retorno, encerrando o processo.

Pitch do Protótipo de Média Fidelidade

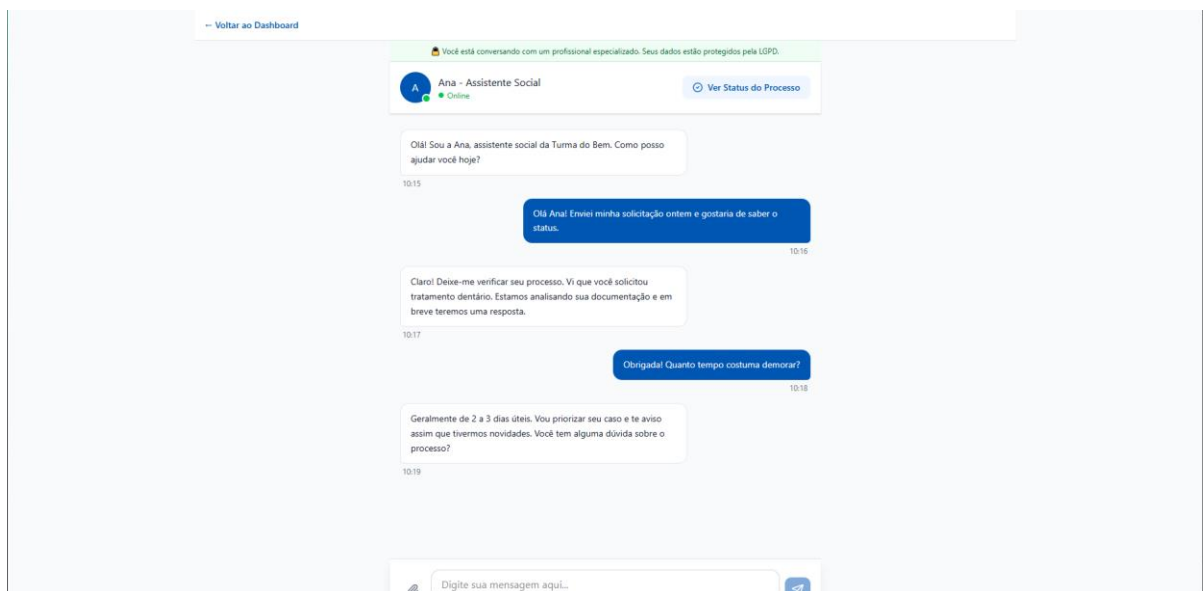
Foi desenvolvido um pitch demonstrando o protótipo do sistema, com foco na experiência do usuário.

[\(Link do vídeo aqui\)](#)

DOMAIN DRIVEN DESIGN USING JAVA



Esta é a tela principal do protótipo, a dashboard onde o usuário consegue visualizar todas as informações cruciais sobre cada cliente da Turma do Bem e acessar mais informações se precisar.



Esta é a tela de comunicação do cliente da Turma do Bem com os funcionários da ONG, uma simples tela onde o usuário pode receber e enviar mensagens.

[Voltar ao Dashboard](#)

Central de Comunicação - Funcionários

Minha Caixa de Entrada

Todos

Beneficiários

Dentistas

Clinicas

LO

Lucas Oliveira

Beneficiário

Obrigado pela ajuda! Quando posso ag...

10:45

MD

Dra. Mariana Santos

Dentista

Posso atender na quinta-feira à tarde

09:30

JC

Juliana Costa

Beneficiário

Enviei os documentos solicitados

Ontem

CS

Clinica Sorrir Bem

Clinica

Temos 5 vagas disponíveis este mês

Ontem

RS

Roberto Silva

Beneficiário

Bom dia! Há alguma novidade?

14/03

LO

Conversando com: Lucas Oliveira (Beneficiário)

Acolhimento Alberto #123

Olá Lucas! Vi sua solicitação e estou aqui para ajudar. Como você está se sentindo hoje?

10:30

Olá Ana! Estou com muita dor no dente ainda. Conseguiu encontrar algum dentista para me ajudar?

10:32

Sim! Encontrei uma dentista maravilhosa, a Dra. Mariana, que pode te atender esta semana. Ela é especialista em tratamento de canal.

10:35

Que ótima notícia! Muito obrigado mesmo. Vocês estão mudando minha vida.

10:40

Fico muito feliz em poder ajudar! Vou te mandar os detalhes do agendamento em alguns minutos.

10:42

Obrigado pela ajuda! Quando posso agendar?

10:45

Enviar

LO

Lucas Oliveira

Beneficiário

(11) 98765-4321

São Paulo, SP

Pedido de Ajuda enviado em 15/05/2026

Pai de família, desempregado, necessita tratamento de canal urgente. Dor crônica há 3 meses.

Ver Processo Completo

Finalizar Atendimento

Aqui a mesma tela de comunicação entre funcionário da Turma do Bem e seus clientes, porém desta vez do ponto de visão do funcionário da ONG, ele pode selecionar outros clientes, filtrar entre eles, e visualizar todo o atendimento em (Ver Processo Completo) o que se relaciona com o método de selecionar o Atendimento no projeto Java.

23



Análise de Pedido - Maria Silva Santos (Beneficiária)

Acolhimento Aberto #123

Dados Pessoais

NOME COMPLETO	IDADE
Maria Silva Santos	40 anos
CPF	TELEFONE
123.456.789-00	(11) 98765-4321
DATA DE NASCIMENTO	E-MAIL
15/06/1985	maria.silva@email.com
ENDEREÇO	
Rua Flores, 123, São Paulo, SP	
STATUS DE VULNERABILIDADE	
Desempregada, mãe de 3, renda total < 1 salário mínimo	

Contexto Clínico

TRATAMENTO SOLICITADO

Dores agudas constantes, falta de dentes frontais

HISTÓRIA DETALHADA

"Sinto muita vergonha de sorrir e falar em público, perdi 4 dentes na frente em um acidente há 5 anos e não consigo tratamento. Não consigo emprego por causa disso e sinto muita dor. Meus filhos merecem uma mãe que possa sorrir sem medo. Preciso muito dessa ajuda para reconstruir minha vida e autoestima."

→ Encaminhar Formulário

↓ Baixar Formulário

↻ Sincronizar e Enviar para o Salesforce e Aprovar

Essa tela é onde o funcionário da ONG consegue visualizar mais sobre o atendimento do cliente, de novo se relacionando com o método de selecionar o Atendimento no projeto Java.

O protótipo de alta fidelidade desenvolvido no Figma representa a camada de Visão (View) do sistema e foi estruturado para operar sobre as regras de negócio implementadas no backend Java.

- **Interface de Gestão:** O Dashboard centraliza métricas e listagens que são alimentadas pelos métodos de consulta das classes BeneficiarioService e AtendimentoService.
- **Fidelidade às Entidades:** Cada campo de entrada no protótipo (como CPF, CRO e Status de Vulnerabilidade) corresponde rigorosamente aos atributos definidos nas classes do pacote br.com.fiap.entities.
- **Aplicação de Regras de Negócio:**
 - **Triagem:** A classificação visual de prioridade no Figma é o resultado da lógica do método definirPrioridade().
 - **Validação:** O fluxo de cadastro do protótipo depende das validações de dados (CPF e campos obrigatórios) codificadas na camada de serviço.
- **Persistência de Dados:** As interações do usuário (salvar, editar ou excluir) acionam os objetos DAO, garantindo a sincronia entre a interface gráfica e o banco de dados Oracle.

1. Descrição do Projeto e Funcionalidades

O sistema Sorriso Conectado é uma plataforma de gestão desenvolvida para a ONG Turma do Bem. O foco principal desta entrega é a automatização do fluxo de atendimento e o gerenciamento das entidades principais (Beneficiários, Dentistas, Clínicas e Voluntários).

Funcionalidades Implementadas:

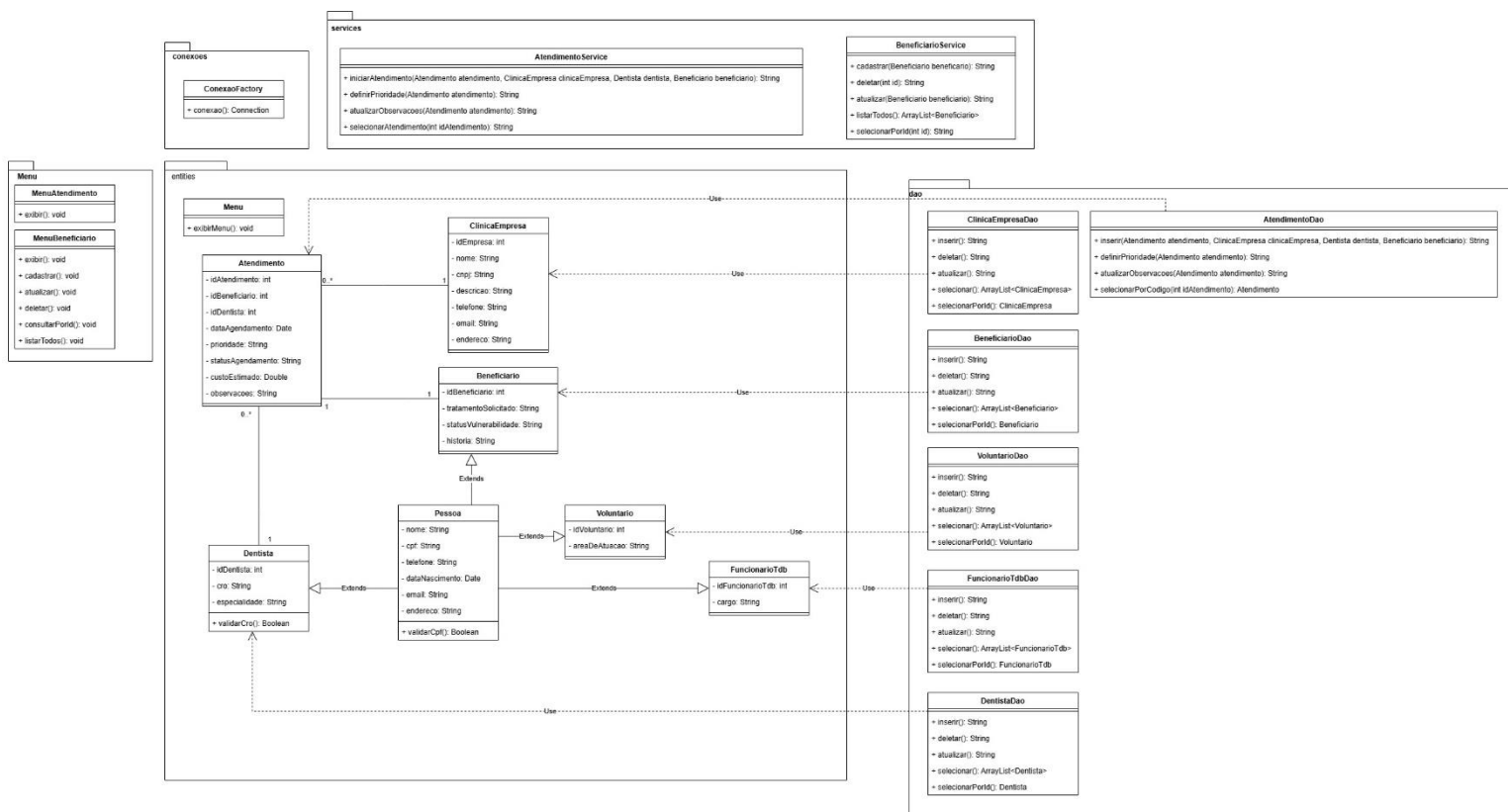
Gestão de Beneficiários (CRUD Completo): Permite cadastrar, consultar, atualizar e excluir pessoas que solicitam tratamento.

Gestão de atendimentos: Controle de triagem, definição de prioridade automática e atualização de observações clínicas.

Validação de Regras de Negócio: Sistema de validação de CPF e verificação de integridade de dados (como custo de agendamento não negativo).

Interface Interativa: Menu dinâmico via JOptionPane com submenus organizados por categoria.

2. Diagrama de Classes (UML)



Descrição da Estrutura:

Herança: Utilizamos a classe Pessoa como classe base para Beneficiario, Dentista, Voluntario e FuncionarioTdb, garantindo o reuso de atributos como Nome, CPF e Endereço. **Associações:** A classe Atendimento atua como o ponto central, associando-se com IDs de Beneficiários, Dentistas e Clínicas.

3. Descrição dos Métodos e Regras de Negócio

Para atender aos requisitos da Sprint, destacamos 4 métodos principais com lógica de negócio:

validarCpf(String cpf) (Classe Pessoa): Implementa a lógica de limpeza de caracteres especiais e verificação de comprimento, impedindo cadastros com dados corrompidos.

iniciarAtendimento() (Classe AtendimentoService): Valida se os custos são positivos e se todos os IDs relacionados existem antes de persistir no banco. Define o status "Novo" por padrão.

definirPrioridade() (Classe AtendimentoService): Garante que um atendimento não fique sem classificação de urgência, essencial para a fila de espera da ONG.

listarTodos() (Classe BeneficiarioService): Implementa a construção dinâmica de relatórios usando StringBuilder e formatação de texto para exibição ao usuário.

4. Camada de Persistência (DAO)

O sistema utiliza o padrão DAO (Data Access Object) para isolar a lógica de banco de dados.

Tecnologia: JDBC com Oracle Database.

Operações: Cada entidade possui métodos inserir, atualizar, deletar e selecionar, garantindo que a regra de negócio (Service) nunca manipule o SQL diretamente.

5. Instruções para Execução

Para rodar o projeto, siga os passos abaixo:

Pré-requisitos: Possuir o JDK 17 ou superior instalado e o driver JDBC da Oracle (ojdbc8.jar).

Configuração: Importe o projeto em uma IDE (IntelliJ ou Eclipse). Verifique se o arquivo Crip.properties ou sua classe de conexão possui as credenciais corretas do banco de dados FIAP.

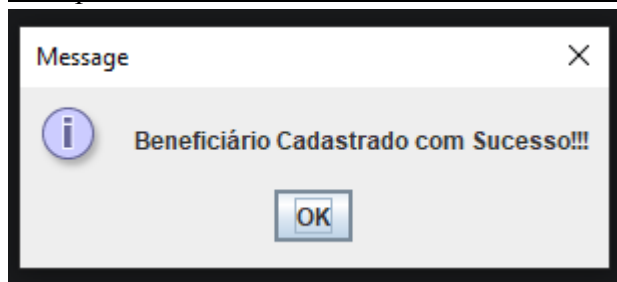
Execução: Localize a classe br/com/fiap/main/main.java

Uso: O sistema abrirá uma janela de diálogo. Utilize os números de 1 a 7 para navegar entre os módulos de Beneficiários e Atendimentos.

6. Prints dos Testes (Resultados)

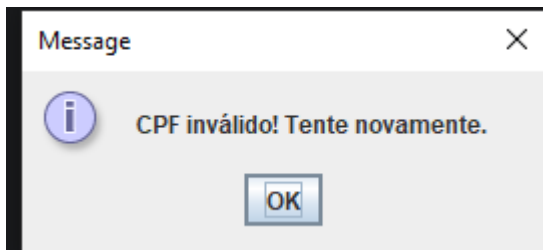
(Nesta parte, você deve tirar prints das janelas do JOptionPane funcionando)

- Exemplo 1: Cadastro de Beneficiário com sucesso.

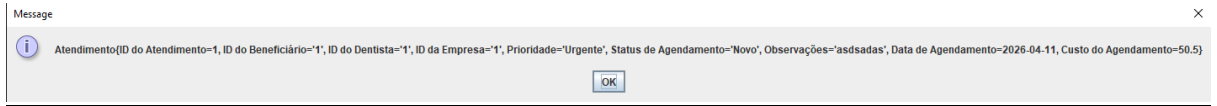


ID_BENEFICIARIO	NOME_BENEFICIARIO	CPF_BENEFICIARIO	NASC_BENEFICIARIO	FONE_BENEFICIARIO	EMAIL_BENEFICIARIO	END_BENEFICIARIO	TRAT_BENEFICIARIO	STT_BENEFICIARIO	HIST_BENEFICIARIO
1	kenji	48032191844	25/07/06	11940798318	(null)	aaaa	bbbb	cccc	dddd
2	9 Yuri Alberto	99999999999	01/01/99	11999999999	yuri.alberto@corinthians.com	Corinthians	Teste	Teste	Teste
3	50 awdwda	14714714714	25/07/06	11940798319	wdwadv	efefe	dwawadwa	wadwadwa	dwadwadwa

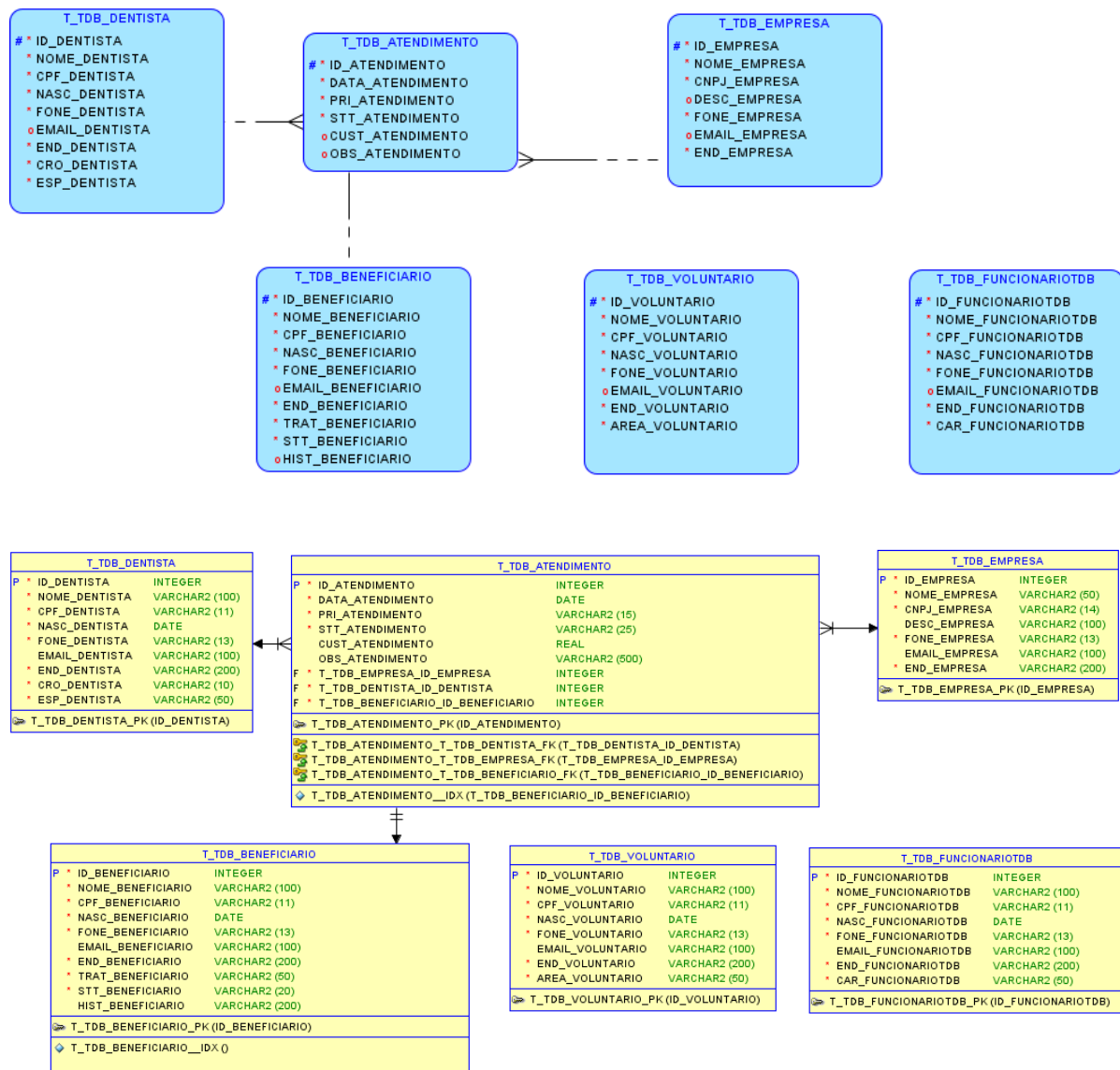
- Exemplo 2: Erro ao tentar cadastrar CPF inválido.



- Exemplo 3: Listagem de atendimentos recuperada do banco de dados.



BUILDING RELATIONAL DATABASE



A evolução do projeto Sorriso Conectado nesta sprint focou na transição do modelo lógico para o físico, utilizando scripts DDL no banco de dados Oracle para garantir a integridade dos dados da ONG Turma do Bem.

MODELO DESCRITIVO (DICIONÁRIO DE DADOS)

Este modelo detalha as tabelas implementadas no banco de dados Oracle para o ecossistema Sorriso Conectado.

- T_TDB_BENEFICIARIO:** Armazena o cadastro completo dos pacientes. Possui chaves únicas (UNIQUE) para CPF, E-mail e Telefone, garantindo a integridade dos dados e evitando duplicidade de assistidos.
- T_TDB_VOLUNTARIO:** Tabela base para todos os colaboradores da ONG. Registra as informações fundamentais e a área de atuação de cada voluntário.

- **T_TDB_DENTISTA:** Tabela especializada que armazena os profissionais de saúde. Utiliza o ID_DENTISTA como PK e FK, vinculando-se obrigatoriamente à tabela de voluntários e exigindo o registro do CRO.
- **T_TDB_EMPRESA:** Registro das clínicas e empresas parceiras onde ocorrem os atendimentos. Gerencia dados corporativos validados por CNPJ.
- **T_TDB_FUNCIONARIOTDB:** Cadastro dos colaboradores administrativos internos da instituição.
- **T_TDB_ATENDIMENTO:** Entidade central do sistema. Consolida a transação entre Beneficiário, Dentista e Clínica. Possui regras de validação (CHECK Constraints) para assegurar que os campos de **Status** e **Prioridade** sigam estritamente o fluxo operacional da Turma do Bem.

1. Estrutura e Normalização

O banco foi estruturado em tabelas específicas (T_TDB_BENEFICIARIO, T_TDB_DENTISTA, T_TDB_ATENDIMENTO, etc.) seguindo a 3ª Forma Normal. Diferente do Java, onde há herança via classe Pessoa, no banco optamos pela especialização com chaves primárias e estrangeiras compartilhadas, otimizando a performance das consultas realizadas pelos DAOs.

2. Integridade e Regras de Negócio

Foram implementadas camadas de proteção diretamente no SQL:

- Constraints de CHECK: Validam domínios de dados, permitindo apenas status e prioridades pré-definidos (ex: 'Urgente', 'Novo').
- Constraints UNIQUE: Impedem a duplicidade de CPFs, CNPJs, E-mails e Telefones.
- Tipagem Precisa: Uso de VARCHAR2 para textos, DATE para registros temporais e NUMBER(10,2) para controle de custos operacionais.

3. Relacionamentos

A tabela T_TDB_ATENDIMENTO atua como o ponto central de integração. Através de Chaves Estrangeiras (FKs), o sistema garante o vínculo obrigatório entre um Beneficiário, um Dentista e uma Clínica Empresa, assegurando a rastreabilidade total de cada procedimento gerido pela plataforma.

SPRINT 4

SOFTWARE ENGINEERING AND BUSINESS MODEL

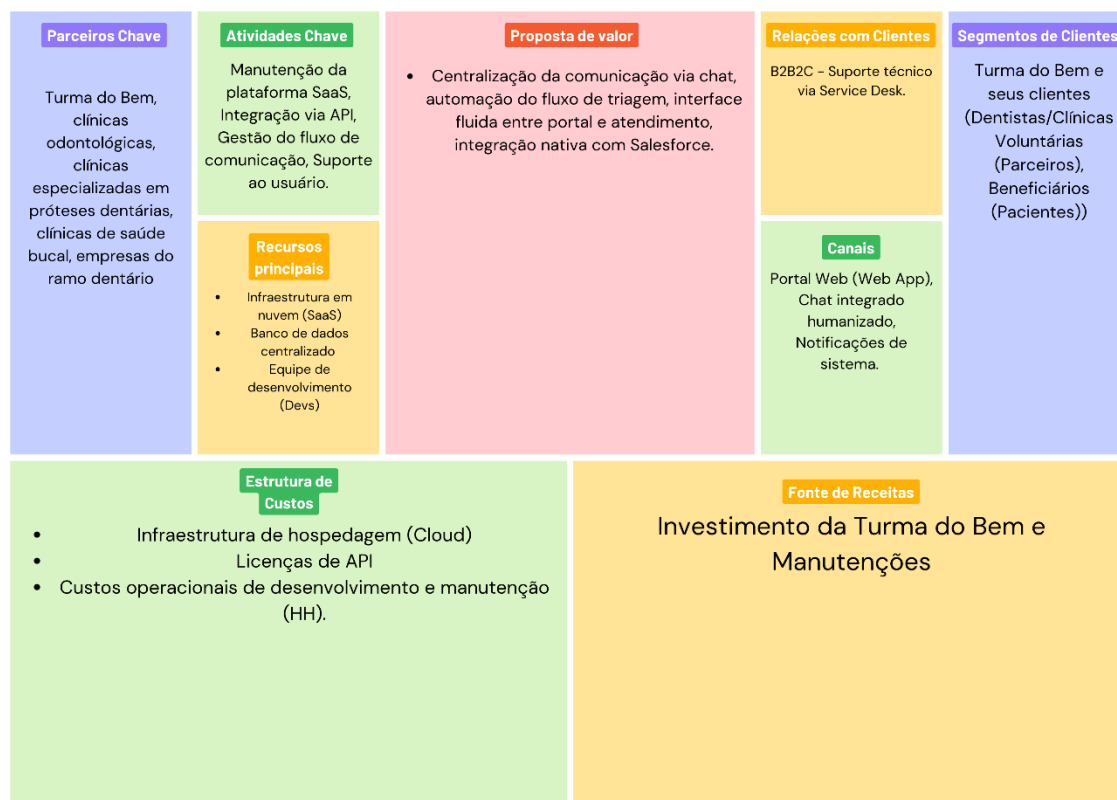
1. Sobre o Negócio

- **Oportunidade:** A Turma do Bem já possui um sistema Salesforce, mas ainda falta uma camada de acesso aos clientes. A oportunidade está em concentrar toda a operação: ao invés da equipe central da ONG mediar cada passo, entrando em contato com cada cliente manualmente por vários meios diferentes, a Turma do Bem com o Sorriso Conectado terá um portal de comunicação com o dentista, beneficiário e clínicas, enquanto o sistema Salesforce que eles já possuem permanece como o meio de operação para gerenciar financeiro, triagens e outras funções que não são e não dependem da comunicação direta ao cliente, essas funções ficaram concentradas no sistema do Sorriso Conectado.
- **Modelo de Relacionamento:** O modelo de relacionamento da solução é B2B2C, atuando como uma plataforma de orquestração que permite à Turma do Bem (B) gerir com eficiência sua rede de dentistas e clínicas parceiras (B), garantindo que o serviço de saúde chegue com agilidade ao beneficiário final (C). Nesse fluxo, a plataforma atua como o elo centralizador que viabiliza a prestação do serviço odontológico social, conectando a organização aos prestadores e assegurando que o paciente receba o atendimento necessário dentro dos padrões e regras estabelecidos pela ONG.
- **Tipo de Inovação:** Inovação de Integração. O Sorriso Conectado não substituirá o ecossistema da TdB, e sim criará um "braço" operacional que estende o atual sistema da ONG, o Salesforce, para quem realmente realiza o atendimento, facilitando a comunicação e a intermediação de dados entre plataforma de contato e o atual sistema de gestão da TdB.
- **Modelo de Negócio:** O modelo de negócios da startup baseia-se em Software como Serviço (SaaS), atuando como o principal canal de comunicação e conexão da Turma do Bem. Em vez de depender de e-mails dispersos ou contatos manuais, a nossa plataforma unifica todos os envolvidos em um único ambiente digital. O sistema atua como uma "ponte" ativa: ele recebe o beneficiário no portal, organiza as informações de triagem e direciona o caso para um chat integrado, onde o diálogo com a equipe da ONG acontece de forma fluida e humanizada. O valor da nossa startup está em transformar uma rede de contatos complexa em uma conversa simples e organizada, garantindo que dentistas, parceiros e pacientes estejam sempre conectados e alinhados, sem os ruídos da comunicação tradicional.

Business Model Canva

Business Model Canvas

Sorriso Conectado



2. Mercado e Competidores

- **Mercado Alvo:** O mercado-alvo do Sorriso Conectado concentra-se em HealthTechs voltadas ao impacto social, com foco na orquestração de redes de voluntariado odontológico. O público-alvo é composto pela Turma do Bem, seus dentistas parceiros e os beneficiários finais, nicho que demanda ferramentas capazes de integrar triagem e atendimento em um fluxo de trabalho especializado, algo que softwares de gestão clínica convencionais não oferecem.
- **Análise de Concorrentes:** No campo competitivo, o Sorriso Conectado enfrenta a concorrência de ferramentas genéricas, como WhatsApp e planilhas, que, embora gratuitas, fragmentam a comunicação e perdem o histórico do paciente. O Sorriso Conectado compete também com softwares de gestão comercial que ignoram as particularidades do atendimento voluntário, o que torna a experiência operacional lenta e burocrática para a ONG.
- **Diferenciais:** O diferencial competitivo do Sorriso Conectado é a integração *omnichannel* com o Salesforce, que elimina a redundância de dados ao transferir o contexto do portal web diretamente para o chat humanizado. Ao transformar a comunicação em um processo inteligente e sem atritos, a solução reduz drasticamente o tempo administrativo da equipe, permitindo que a Turma do Bem escale seu impacto social com agilidade, sem perder a personalização do atendimento.

3. Precificação da Solução

1. Custo Operacional Mensal da Equipe

- Integrante 1: R\$ 4.075,10
- Integrante 2: R\$ 3.950,00
- Integrante 3: R\$ 3.512,55
- Custo Mensal Total (Equipe): R\$ 11.537,65

2. Cálculo da Hora Técnica (HH)

- Horas Totais Disponíveis: $110h \times 3 = 330$ horas/mês
- Custo Base por Hora: $R\$ 11.537,65 \div 330h = R\$ 34,96/hora$
- Margem de Startup (100%): Adicionada para custos operacionais (nuvem, licenças, reserva):
 - *Cálculo:* $R\$ 34,96 \times 2 = R\$ 69,92$
- Hora Técnica Definida (Arredondada): R\$ 70,00/hora

3. Preço Final do Projeto (MVP)

Considerando o esforço estimado de 300 horas para o desenvolvimento total:

- Cálculo do Preço: $300h \times R\$ 70,00/hora$
- Preço Final ao Cliente: R\$ 21.000,00

4. Retorno do Investimento (ROI)

Para chegar ao cálculo, quantificamos os seguintes indicadores de eficiência:

- **Redução do Tempo de Triagem:** Estimativa de economia de 15 minutos por atendimento administrativo, considerando que a automação via portal elimina a digitação manual de formulários.
- **Aumento da Capacidade Operacional:** Com o fluxo automatizado, a equipe da ONG consegue processar um volume 20% maior de pacientes no mesmo período, sem necessidade de novas contratações.
- **Redução de "Churn" de Voluntários:** A facilidade de comunicação reduz a desistência de dentistas, economizando o custo de prospecção e treinamento de novos voluntários (estimado em R\$ 1.500,00 por voluntário substituído).
- **Economia Operacional Direta:** O ganho financeiro consolidado mensal (tempo de equipe + redução de retrabalho + eficiência da rede) foi estabelecido em R\$ 5.000,00.

Racional do Cálculo do ROI

- **Investimento Inicial (Custo do Projeto):** R\$ 21.000,00.
- **Ganho Líquido Anual:** Consideramos a economia de R\$ 5.000,00 mensais por 12 meses, totalizando R\$ 60.000,00 de benefício econômico no primeiro ano.

Cálculo do ROI:

$$\% ROI = \left(\frac{Receita - Custo}{Custo} \right) \times 100$$

$$\% ROI = \left(\frac{60.000 - 21.000}{21.000} \right) \times 100 = 185,7\%$$

O projeto apresenta um ROI de 185,7% e se pagará em 4,2 meses (Payback).

5. Acordo de Nível de Serviço (SLA)

1. Serviços Prestados

- **Gestão de Infraestrutura:** Garantia de hospedagem e disponibilidade do portal web e do sistema de chat integrado.
- **Sincronização de Dados:** Manutenção das APIs de integração entre o portal e o Salesforce.
- **Suporte Técnico:** Atendimento às demandas de uso, correção de erros (bugs) e suporte ao primeiro nível para comunicadores da ONG.

2. Níveis de Serviço

O tempo de resposta e solução será categorizado conforme o impacto operacional:

- **Crítico (Sistema Indisponível):** Resposta em até 4 horas úteis; solução em até 8 horas úteis.
- **Médio (Erro em Funcionalidades/Chat):** Resposta em até 8 horas úteis; solução em até 24 horas úteis.
- **Baixo (Dúvidas/Melhorias):** Resposta em até 24 horas úteis; solução em até 3 dias úteis.

3. Disponibilidade do Serviço

- **Janela de Disponibilidade:** A plataforma será mantida em regime de 24 horas por dia, 7 dias por semana (24/7).
- **Monitoramento:** A equipe de desenvolvimento realizará manutenções preventivas preferencialmente fora do horário comercial (das 22h às 06h) ou aos finais de semana, com aviso prévio de 48 horas via sistema.
- **Disponibilidade Garantida:** A meta de disponibilidade mensal (uptime) é de 98,5%.

4. Requisitos de Help Desk/Service Desk

O suporte será centralizado em um canal único para facilitar o rastreamento dos chamados:

- **Canal de Atendimento:** O usuário reportará problemas diretamente através de um painel de "Service Desk" integrado ao sistema ou via e-mail oficial de suporte.
- **Registro e Rastreabilidade:** Todos os chamados receberão um número de protocolo, permitindo que a Turma do Bem acompanhe o status da resolução em tempo real.
- **Escalonamento:** Caso um erro não seja resolvido no prazo acordado, o chamado será automaticamente escalado para o nível de liderança técnica da startup.

5. Política de Multas e Descontos

Como forma de garantir o compromisso com a qualidade, foi adotado uma política de crédito (abatimento) proporcional ao descumprimento do nível de disponibilidade:

- **Uptime entre 95% e 98,4%:** Desconto de 5% no valor da mensalidade de manutenção do mês subsequente.
- **Uptime abaixo de 95%:** Desconto de 15% no valor da mensalidade de manutenção do mês subsequente.
- **Isenção:** Não haverá aplicação de multas caso a indisponibilidade seja causada por falhas em serviços de terceiros (como a própria indisponibilidade da API do Salesforce ou provedores de nuvem) ou por eventos de força maior.

DOMAIN DRIVEN DESIGN USING JAVA

Repositório do projeto no github: <https://github.com/KenjiFW13/projeto-java-sorriso-conectado-quarkus>

1. Atualizações

Nessa sprint foram implementadas as funcionalidades de ligação do projeto Java ao Front-end utilizando o Framework Quarkus com as APIs Rest e JDBC Oracle, então foi adicionado ao projeto Java as camadas BO e Resource, e as camadas services e menu foram removidas do projeto por se mostrarem obsoletas nessa versão. Alguns dos métodos que anteriormente se encontravam nas camadas removidas, foram implementadas diretamente na camada BO e utils.

```
public static boolean validarCpf(String cpf) { 14 usages  kenjiFW13
    if (cpf == null) return false;
    String apenasNumeros = cpf.replace(".", "").replace("-", "");
    return apenasNumeros.length() == 11;
}

public static boolean validarCro(String cro){ 2 usages  kenjiFW13
    if (cro == null) return false;
    String apenasNumeros = cro.replace(".", "").replace("-", "");
    return apenasNumeros.length() == 7;
}

public static boolean validarCnpj(String cnnpj){ 1 usage  kenjiFW13
    if (cnnpj == null) return false;
    String apenasNumeros = cnnpj.replace(".", "").replace("-", "").replace("/", "");
    return apenasNumeros.length() == 14;
}

private static final String EMAIL_REGEX = "[a-zA-Z0-9_+&*-]+(?:\\.[a-zA-Z0-9_+&*-]+)*@(?:[a-zA-Z0-9-]+\\.)+[a-zA-Z]{2,7}$"; 1 usage
private static final Pattern PATTERN = Pattern.compile(EMAIL_REGEX); 1 usage

public static boolean validarEmail(String email) { 9 usages  kenjiFW13
    if (email == null || email.isEmpty()) return false;
    return PATTERN.matcher(email).matches();
}
}
```

Com a implementação da classe ValidatorUtil, temos nela 4 métodos de validação para o sistema, que são aplicados nos métodos de Inserts e Updates da camada BO, sendo os seguintes métodos:

- validarCpf, validarCro, validarCnpj: Todos esses métodos seguem a mesma lógica, recebem um parâmetro para ser validado, e para isso passam pela

verificação de tamanho e limpeza de campo, ou seja são removidos caracteres que podem interferir diretamente no tamanho do parâmetro esperado

- `validarEmail`: Este método segue a mesma lógica dos outros métodos, porém seu processo de validação é um pouco diferente, neste método utilizamos as funções do `Regex` e do `Pattern` para poder validar o email, já que todo email segue um padrão (exemplo@exemplo.exemplo). O `Regex` já havia sido ensinado para os alunos da TDS na matéria de Inteligência Artificial, e se apresentou como uma opção de implementação para o sistema.

```
10 public class BeneficiarioBO { 3 usages kenjifw13 *
11
12     BeneficiarioDao beneficiarioDAO; 5 usages
13
14     // Inserir
15 @ public String inserirBO(Beneficiario beneficiario) throws SQLException, ClassNotFoundException { 1 usage kenjifw13
16         BeneficiarioDao beneficiarioDAO = new BeneficiarioDao();
17
18         if (!ValidatorUtil.validarCpf(beneficiario.getCpf())){
19             return "Erro: Cpf inválido!";
20         }
21
22         if (beneficiario.getIdBeneficiario() <= 0) {
23             return "Erro: O ID do beneficiário deve ser maior que zero!";
24         }
25
26         if (beneficiario.getTratamentoSolicitado() == null || beneficiario.getTratamentoSolicitado().isEmpty()) {
27             return "Erro: O tratamento solicitado precisa ser informado!";
28         }
29
30         if (!ValidatorUtil.validarEmail(beneficiario.getEmail())){
31             return "E-mail inválido!";
32         }
33
34         return beneficiarioDAO.inserir(beneficiario);
35     }
36
37     // Deletar
38 public void deletarBO(int codigo) throws SQLException, ClassNotFoundException { 1 usage kenjifw13
39         BeneficiarioDao beneficiarioDAO = new BeneficiarioDao();
40         beneficiarioDAO.deletar(codigo);
41     }
42
43     // Update
44 @ public String atualizarBO(Beneficiario beneficiario) throws SQLException, ClassNotFoundException { 1 usage kenjifw13
45         BeneficiarioDao beneficiarioDAO = new BeneficiarioDao();
46
47         if (!ValidatorUtil.validarCpf(beneficiario.getCpf())){
```

A camada BO foi adicionada com os métodos de inserção, deletar, atualizar, seleção e seleção por id por padrão, todos os métodos “puxando” do método original aplicado na camada DAO.

```
// Exibir história
public String exibirHistoria(int codigo) throws SQLException { 1 usage  ⚡ kenjifw13
    Beneficiario beneficiario = beneficiarioDao.selecionarPorCodigo(codigo);

    if (beneficiario == null) {
        return "Beneficiário não encontrado!";
    }

    if (beneficiario.getHistoria() == null || beneficiario.getHistoria().isEmpty()) {
        return "Beneficiário sem história cadastrada!";
    }

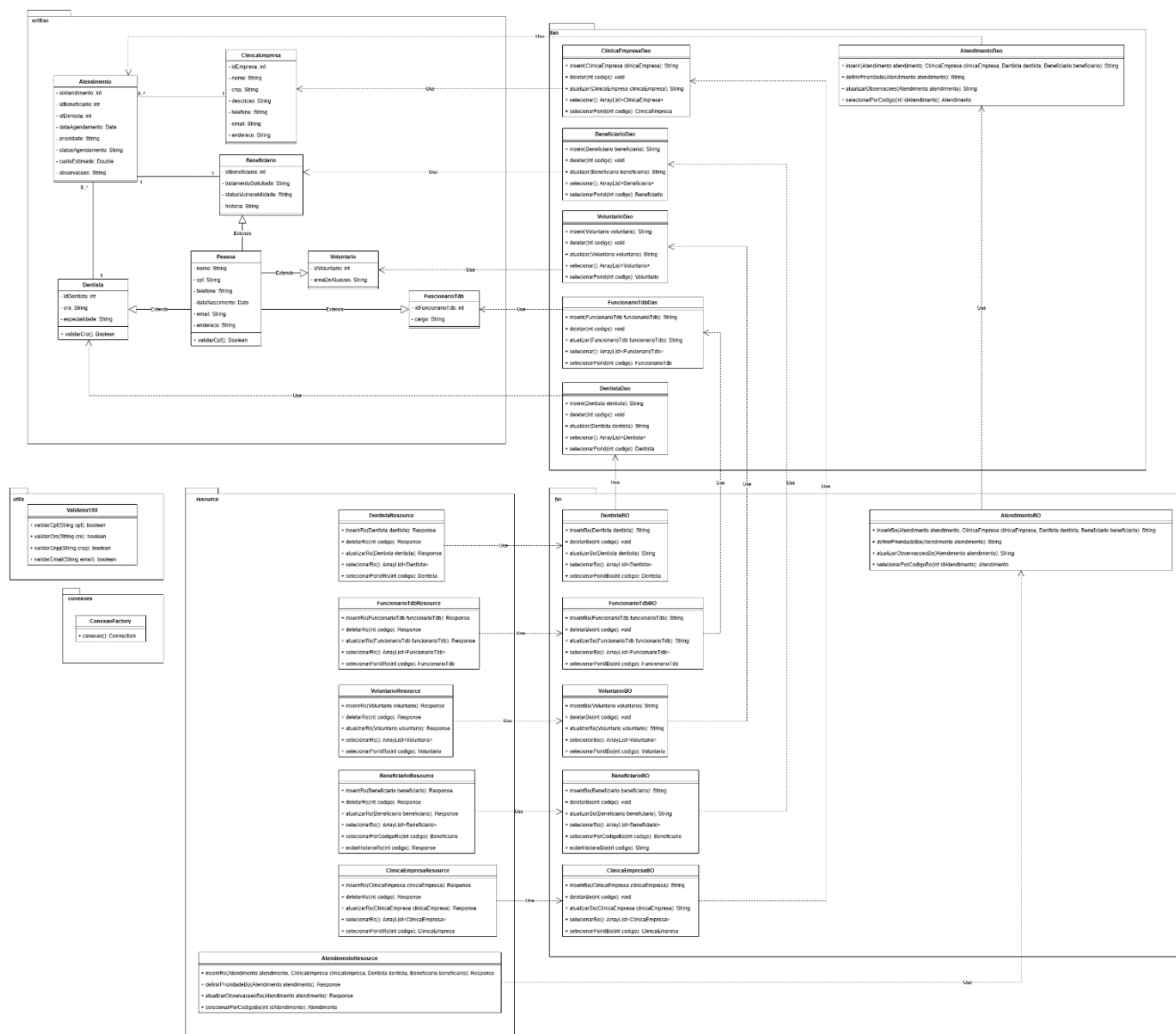
    return "--- História do Beneficiário: " + beneficiario.getNome() + ": \n" + beneficiario.getHistoria();
}
}
```

O método `exibirHistoria()` na classe `BeneficiarioBO` serve para exibir apenas a história de um beneficiário assim que o mesmo é buscado e selecionado por um código Id.

```
13  @Provider  ⚡ kenjifw13
14
15  @Path("/{beneficiario}")
16  public class BeneficiarioResource {
17
18      private BeneficiarioBO beneficiarioBO = new BeneficiarioBO(); 6 usages
19
20      // Inserir
21      @POST  ⚡ kenjifw13
22      @Consumes(MediaType.APPLICATION_JSON)
23      http://localhost:8080/beneficiario
24      public Response inserirRs(Beneficiario beneficiario, @Context UriInfo uriInfo) throws SQLException, ClassNotFoundException {
25          beneficiarioBO.inserirBO(beneficiario);
26          UriBuilder builder = uriInfo.getAbsolutePathBuilder();
27          builder.path(Integer.toString(beneficiario.getIdBeneficiario()));
28          return Response.created(builder.build()).build();
29      }
30
31      // Deletar
32      @DELETE  ⚡ kenjifw13
33      @Path("/{codigo}")
34      @Consumes(MediaType.APPLICATION_JSON)
35      http://localhost:8080/beneficiario/{codigo}
36      public Response deletarRs(@PathParam("codigo") int codigo) throws SQLException, ClassNotFoundException {
37          beneficiarioBO.deletarBO(codigo);
38          return Response.ok().build();
39      }
40
41      // Update
42      @PUT  ⚡ kenjifw13
43      @Path("/{codigo}")
44      @Consumes(MediaType.APPLICATION_JSON)
45      http://localhost:8080/beneficiario/{codigo}
46      public Response atualizarRs(Beneficiario beneficiario, @PathParam("codigo") int codigo) throws SQLException, ClassNotFoundException {
47          beneficiarioBO.atualizarBO(beneficiario);
48          return Response.ok().build();
49      }
50
51      // Selecionar
52      @GET  ⚡ kenjifw13
53      @Produces(MediaType.APPLICATION_JSON)
```

A camada Resource foi adicionada, onde foi implementado os verbos HTTP da API Restful ao sistema (POST, PUT, GET, DELETE), essa parte é a responsável por fazer a conexão de banco de dados e Back-end ao Front-end. No Front-end do sistema, é possível interagir e visualizar os dados que estão no banco de dados e passam pelo back-end.

2. Diagrama de Classe



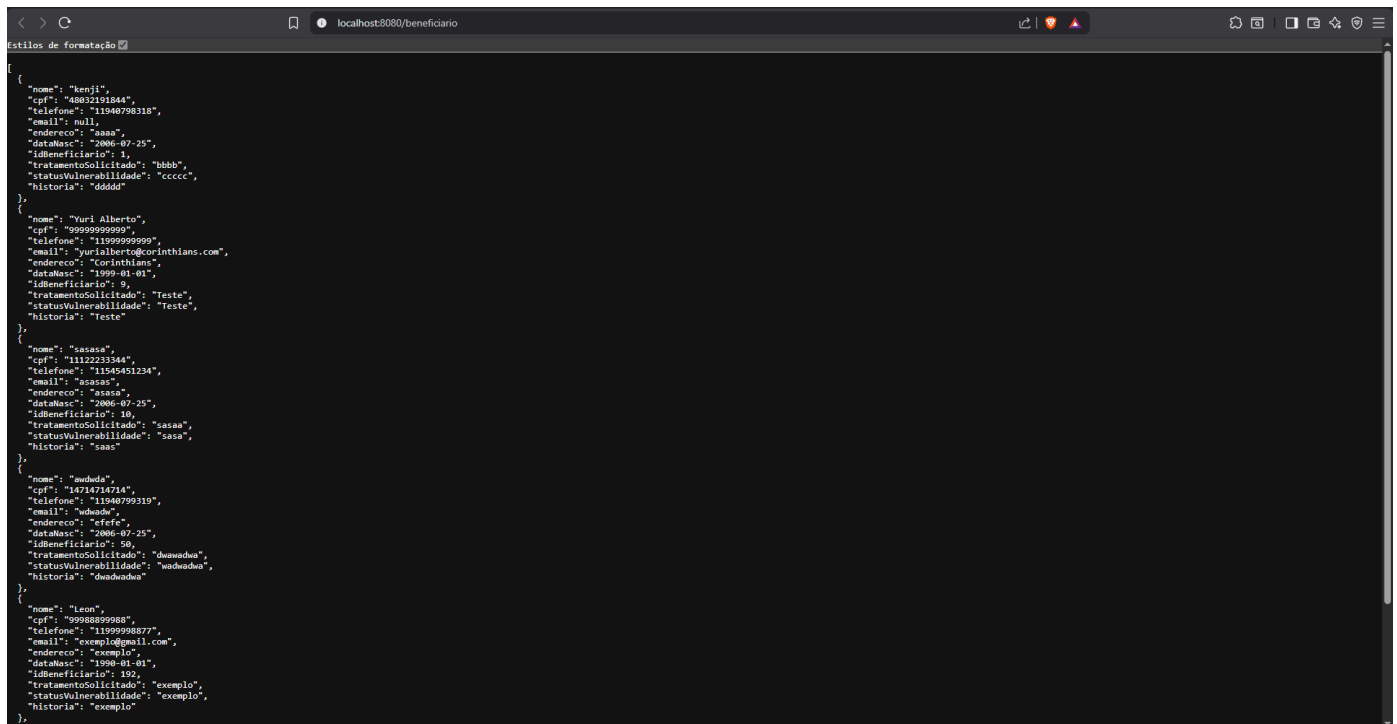
Este é o novo diagrama de classe do projeto Java, é possível visualizar a remoção das camadas Services e Menu e a implementação das camadas Resource e BO que se conectam entre si e BO se conectando diretamente com a camada DAO pela sua dependência direta com a camada.

3. Instruções para executar

Para rodar o projeto, siga os passos abaixo:

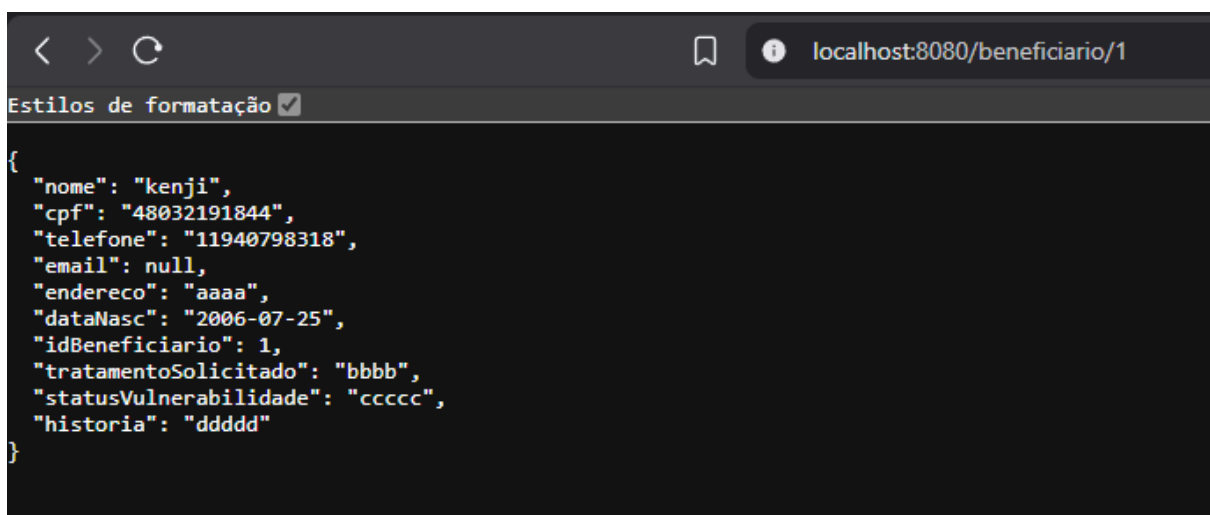
1. **Pré-requisitos:** Possuir o JDK 21 ou superior instalado, driver JDBC da Oracle (ojdbc8.jar) e o plugin Quarkus Tools.
2. **Configuração:** Importe o projeto em uma IDE (IntelliJ ou Eclipse). Verifique se a classe de conexão (ConnectionFactory) possui as credenciais corretas do banco de dados.
3. **Execução:** Pelo IntelliJ, apenas rodar o sistema Quarkus ou pelo terminal escrever `mvn quarkus:dev`
4. **Uso:** O sistema se executado corretamente, irá abrir um servidor local na porta 8080, o qual mostrará a interface do Quarkus e o back-end do projeto conectado

ao banco de dados, os dados são exibidos pelo formato JSON. Também é possível subir o projeto na nuvem, como por exemplo o Render, e o servidor rodará corretamente, fique apenas atento aos Endpoints.



```
[{"nome": "kenji", "cpf": "48032191844", "telefone": "11940798318", "email": null, "endereco": "aaaa", "dataNasc": "2006-07-25", "idBeneficiario": 1, "tratamentoSolicitado": "bbbb", "statusVulnerabilidade": "cccc", "historia": "ddddd"}, {"nome": "Yuri Alberto", "cpf": "99999999999", "telefone": "1999999999", "email": "yuri.alberto@corinthians.com", "endereco": "Corinthians", "dataNasc": "1999-01-01", "idBeneficiario": 9, "tratamentoSolicitado": "Teste", "statusVulnerabilidade": "Teste", "historia": "Teste"}, {"nome": "sasasa", "cpf": "11122233344", "telefone": "11545451234", "email": "ssasasa", "endereco": "asasa", "dataNasc": "2006-07-25", "idBeneficiario": 10, "tratamentoSolicitado": "sasaa", "statusVulnerabilidade": "sasa", "historia": "saas"}, {"nome": "muhada", "cpf": "14714714714", "telefone": "11940799319", "email": "wuwadwa", "endereco": "efffe", "dataNasc": "2006-07-25", "idBeneficiario": 50, "tratamentoSolicitado": "dwadwada", "statusVulnerabilidade": "wadwadwa", "historia": "dwadwadwa"}, {"nome": "leon", "cpf": "00000000000", "telefone": "11999998877", "email": "exemplo@gmail.com", "endereco": "exemplo", "dataNasc": "1990-01-01", "idBeneficiario": 192, "tratamentoSolicitado": "exemplo", "statusVulnerabilidade": "exemplo", "historia": "exemplo"}]
```

Nessa print é possível visualizar como é a saída de dados. Formatada em JSON, os objetos são apresentados pelo método de Select do SQL e é utilizado o verbo @GET da API Restful.



```
{  "nome": "kenji",  "cpf": "48032191844",  "telefone": "11940798318",  "email": null,  "endereco": "aaaa",  "dataNasc": "2006-07-25",  "idBeneficiario": 1,  "tratamentoSolicitado": "bbbb",  "statusVulnerabilidade": "cccc",  "historia": "ddddd"}
```

Colocando uma / e um número ao final do endereço, é possível fazer o select pelo Id do objeto, isso vale para a maioria dos Endpoints do projeto.